ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

TIỂU LUẬN MÔN HỌC
IT5440 – NGUYÊN LÝ VÀ KỸ THUẬT PHÂN TÍCH CHƯƠNG TRÌNH
(Principle and Technique of Program Analysis)

Đề tài 02-KH – Phát hiện lỗi và định vị lỗi (hướng nghiên cứu khoa học)

GROUNDED OR GUESSED: KIỂM CHỨNG BẰNG THỰC THI VIỆC ĐỊNH VỊ LỖI NGỮ NGHĨA BẰNG MÔ HÌNH NGÔN NGỮ LỚN

Giảng viên hướng dẫn: PGS. TS. Huỳnh Quyết Thắng

Nhóm M9: Doãn Văn An – 20252778M (nhóm trưởng)
Tạ Công Nam – 20252640M
Nguyễn Hoàng Hải – 20252583M
Đỗ Xuân Tráng – 20252774M
Hoàng Trung Kiên – 20251333M
Mai Văn Tùng – 20252788M
Nguyễn Đức Tùng – 20252772M

Học kỳ: 20261

Mã nguồn và dữ liệu: `https://github.com/doanan293/grounded-or-guessed`

HÀ NỘI – 2026

Cam kết về tính trung thực

Nhóm M9 cam kết nội dung tiểu luận là kết quả làm việc của nhóm dưới sự hướng dẫn của PGS. TS. Huỳnh Quyết Thắng. Mọi số liệu trong báo cáo được sinh tự động từ mã nguồn và dữ liệu công khai tại `https://github.com/doanan293/grounded-or-guessed`; mỗi con số trong văn bản là một macro được tạo từ tệp dữ liệu tương ứng, không gõ tay.

Phương pháp SemLoc, mã nguồn gốc và bộ dữ liệu SemFault-250 thuộc về nhóm tác giả của bài báo gốc [1] và được dùng theo giấy phép Apache-2.0; mọi phần lấy từ SemLoc đều được ghi nguồn và không được trình bày như đóng góp của nhóm. Hai bài báo còn lại do giảng viên giao [2, 3] và các tài liệu khác được trích dẫn tại chỗ sử dụng.

Theo hướng dẫn của môn học, nhóm có sử dụng công cụ trí tuệ nhân tạo để hỗ trợ đọc tài liệu, viết mã nguồn, chạy thực nghiệm và soạn thảo; mọi kết quả đã được nhóm kiểm tra lại bằng mã nguồn và dữ liệu đi kèm.

Hà Nội, ngày tháng 11 năm 2026

Đại diện nhóm M9 (nhóm trưởng)

Doãn Văn An

Mục lục

Cam kết về tính trung thực **i**

Danh mục từ viết tắt và thuật ngữ **x**

1 Mở đầu **1**

1.1 Đặt vấn đề 1

1.2 Câu hỏi nghiên cứu 2

1.3 Đóng góp chính 2

1.4 Cấu trúc báo cáo 3

2 Tổng quan tài liệu và từ khóa **5**

2.1 Phương pháp tìm và chọn tài liệu 5

2.1.1 Nguồn và tiêu chí chọn 5

2.1.2 Bộ từ khóa 5

2.2 Ba bài báo được giao 6

2.2.1 SemLoc: neo lập luận của LLM vào ràng buộc thực thi được . . . 6

2.2.2 EffiHolmes: định vị chỗ sửa lỗi hiệu năng bằng so sánh hai lần chạy 7

2.2.3 Khai thác báo cáo sự cố kết hợp LLM trong công nghiệp 8

2.3 Các hướng nghiên cứu liên quan 8

2.3.1 Định vị lỗi dựa trên phổ và đột biến 8

2.3.2 Định vị lỗi bằng mô hình ngôn ngữ lớn 9

2.3.3 Bộ dữ liệu lỗi và tính hợp lệ của đánh giá 9

2.3.4 Nhiễm dữ liệu và tái hiện khi dùng LLM 9

2.4 So sánh và khoảng trống nghiên cứu 10

2.4.1 Bảng so sánh 10

2.4.2 Khoảng trống và hướng của nhóm 10

3 Cơ sở lý thuyết **12**

3.1 Định vị lỗi dựa trên phổ độ phủ 12

3.1.1 Ma trận độ phủ và các công thức nghi ngờ 12

3.1.2 Chỉ số đánh giá và cách xử lý hòa điểm 12

3.2 Lỗi ngữ nghĩa 13

3.2.1 Vì sao độ phủ mất tín hiệu 13

3.2.2 Định nghĩa khách quan bằng độ phủ 13

3.3 SemLoc: phổ vi phạm ràng buộc ngữ nghĩa 14

3.3.1 Quy trình bốn bước 14

3.3.2 Điểm thưởng cho dòng LLM tự đoán 14

3.4 Công cụ thống kê 15

3.4.1 Kiểm định cặp 15

3.4.2 Khoảng tin cậy bootstrap theo cụm và giá trị Shapley 15

4 Phương pháp và thiết kế thực nghiệm **17**

4.1 Câu hỏi nghiên cứu và giả thuyết 17

4.1.1 Ba câu hỏi nghiên cứu 17

4.1.2 Giả thuyết và tiêu chí bác bỏ 17

4.2 Dữ liệu 18

4.2.1 SemFault-250 và cách chia tập 18

4.2.2 SemFault-250 đã sửa: kiểm chứng dòng lỗi chuẩn bằng thực thi . 18

4.2.3 Lỗi thật BugsInPy 19

4.2.4 Biến thể đổi tên định danh 19

4.3 Các phương pháp so sánh 19

4.3.1 Các phương pháp có sẵn 19

4.3.2 CovGate: bổ sung độ phủ vào SemLoc 20

4.3.3 LLM + độ phủ: một prompt duy nhất 20

4.4 Kiểm tra cách đánh giá và phân rã đóng góp 21

4.4.1 Kiểm tra cách đánh giá của SemLoc 21

4.4.2 Phân rã "grounded hay guessed" 21
4.5 Thiết lập thực nghiệm 21
4.5.1 Mô hình và hạ tầng 21
4.5.2 Lặp, kiểm tra sức khỏe lượt chạy và thống kê 22

5 Kết quả thực nghiệm 23
5.1 RQ1 – Tái hiện và kiểm tra cách đánh giá 23
5.1.1 Tái hiện và hai cách chấm điểm 23
5.1.2 Kiểm chứng phản thực tế và lỗi của gói mã công bố 24
5.1.3 Ảnh hưởng của dòng lỗi chuẩn 25
5.2 RQ2 – Cải tiến và đối chứng 25
5.2.1 Một prompt và quy trình nhiều bước 25
5.2.2 Theo loại lỗi và từng thành phần của CovGate 26
5.2.3 Grounded hay guessed: giá trị biên và phân rã 27
5.2.4 Chi phí 28
5.3 RQ3 – Độ bền 29
5.3.1 Lỗi thật BugsInPy 29
5.3.2 Đổi tên định danh 30
5.3.3 Các họ mô hình 31
5.4 Tóm tắt trả lời các câu hỏi nghiên cứu 32

6 Phân tích thất bại và thảo luận 33
6.1 Phân nhóm tự động 33
6.1.1 Sáu nhóm kết quả 33
6.1.2 Cách chọn ca để đọc 34
6.2 Mơ hồ dòng kề cạnh 34
6.2.1 Dòng đoán lệch có hệ thống xuống dưới 34
6.2.2 Quan hệ đầu khối – thân khối và độ nhạy của Acc@1 34

6.3 Năm mẫu nguyên nhân 35
6.3.1 Dòng điều kiện và câu lệnh được bảo vệ 35
6.3.2 Khối lỗi nhiều dòng 35
6.3.3 Bộ phán định chấp nhận bản sửa bù trừ 35
6.3.4 Nguyên nhân và triệu chứng trên cùng luồng dữ liệu 36
6.3.5 Dòng lỗi chuẩn đáng ngờ 36
6.4 Thảo luận 36
6.4.1 Đọc "guessed" cho đúng 36
6.4.2 Khi nào bằng chứng thực thi có ích 37

7 Đóng góp, triết lý khoa học và các mối đe dọa 38
7.1 Các đóng góp khoa học 38
7.1.1 Đo phần "grounded" và phần "guessed" 38
7.1.2 Với lỗi được kiểm thử chạy tới, một prompt có bằng chứng thực thi vượt quy trình nhiều bước 38
7.1.3 Kết quả công bố phụ thuộc vào cách chấm và thành phần không được mô tả 39
7.1.4 SemFault-250 đã sửa 39
7.1.5 Mơ hồ dòng kề cạnh của Acc@1 cấp dòng 39
7.1.6 CovGate, bảng kiểm tra đánh giá và gói tái hiện 40
7.2 Triết lý khoa học 40
7.2.1 Bằng chứng thực thi là trọng tài 40
7.2.2 Đo đóng góp, so với phương pháp đối chứng đơn giản, và chấp nhận bị bác bỏ 40
7.3 Các mối đe dọa đến tính hợp lệ 41
7.3.1 Tính hợp lệ nội tại và tính hợp lệ của cấu trúc 41
7.3.2 Tính hợp lệ ngoại tại 41

8 Kết luận và hướng phát triển **43**

8.1 Kết luận 43

8.2 Hướng phát triển 44

A Hướng dẫn tái hiện và bảng bổ sung **45**

A.1 Bốn mức tái hiện 45

A.1.1 Từ dữ liệu đến bảng 45

A.1.2 Môi trường 45

A.2 Bảng bổ sung 46

A.2.1 Phân rã trên nhật ký công bố 46

A.2.2 Prompt của phương pháp đối chứng LLM + độ phủ 46

Danh sách bảng

2.1 Bộ từ khóa của đề tài 6

2.2 So sánh ba bài báo được giao với hướng của nhóm 10

5.1 Kết quả công bố tính lại theo hai cách chấm (nhật ký của tác giả, 250 chương trình) 24

5.2 RQ2: so sánh trên tập test (Gemini 3.8 Flash, 3 lần chạy; Δ so với SemLoc) 26

5.3 RQ2 theo nhãn lỗi (dòng lỗi chuẩn đã sửa) 27

5.4 Phân rã Shapley độ chính xác của SemLoc (lượt chạy của nhóm, 3 lần) . 28

5.5 RQ3a: 31 lỗi thật youtube-dl (BugsInPy), 3 lần chạy 30

5.6 RQ3b: bản gốc và bản đổi tên định danh (dòng lỗi chuẩn đã sửa, 3 cặp lần chạy) 30

5.7 RQ3c: năm mô hình trên cùng tập con (dòng lỗi chuẩn đã sửa, 3 lần chạy) 31

5.8 Trả lời ba câu hỏi nghiên cứu 32

6.1 Phân nhóm kết quả theo chương trình (dòng lỗi chuẩn đã sửa) 33

A.1 Phân rã Shapley trên nhật ký công bố của tác giả (dòng lỗi chuẩn gốc) . . 46

Danh sách hình vẽ

3.1 Quy trình của SemLoc (theo [1]); khung nét đứt là thành phần nhóm phát hiện trong mã nguồn 14

5.1 Chi phí và độ chính xác (Acc@3) trên cùng tập con, dòng lỗi chuẩn đã sửa; đường nối hai phương pháp của cùng một mô hình 29

Danh mục từ viết tắt và thuật ngữ

Viết tắt / thuật ngữ	Ý nghĩa
Bộ phán định	Quy trình tự động quyết định một dòng có phải dòng lỗi hay không: một bản sửa nhỏ trên dòng đó làm mọi kiểm thử đạt (oracle)
Bootstrap	Lấy mẫu lại có hoàn lại để ước lượng khoảng tin cậy; ở đây lấy mẫu theo chương trình, 10000 lần
Grounded / guessed	Phần thứ hạng do bằng chứng thực thi quyết định / phần do LLM tự đoán
Phương pháp đối chứng	Phương pháp đơn giản dùng để so sánh (baseline)
Prompt (lời nhắc)	Văn bản đầu vào gửi cho LLM
Quy trình nhiều bước	Chuỗi các bước LLM và thực thi nối tiếp nhau (pipeline), như SemLoc
Token	Đơn vị văn bản mà LLM xử lý và tính phí
Acc@k	Tỉ lệ chương trình có dòng lỗi nằm trong k dòng bị nghi ngờ nhất (độ chính xác top-k)
CF	Kiểm chứng phản thực tế (counterfactual verification): vá thử một dòng rồi chạy lại kiểm thử
CI	Khoảng tin cậy (confidence interval), ở đây là khoảng 95% theo bootstrap
CovGate	Cải tiến do nhóm đề xuất: kết hợp độ phủ với điểm của SemLoc
DD	Gỡ lỗi chia đôi (delta debugging)
GT	Dòng lỗi chuẩn (ground truth) của bộ dữ liệu
LLM	Mô hình ngôn ngữ lớn (large language model)
RQ	Câu hỏi nghiên cứu (research question)
SBFL	Định vị lỗi dựa trên phổ độ phủ (spectrum-based fault localization)
SemFault-250	Bộ dữ liệu 250 chương trình Python có lỗi ngữ nghĩa của SemLoc
SemFault-250 đã sửa	Bộ dữ liệu trên với dòng lỗi chuẩn được nhóm kiểm chứng và sửa lại bằng thực thi (tiếng Anh: SemFault-250-repaired)
Shapley	Giá trị Shapley: cách chia một tổng cho các thành phần theo đóng

Chương 1. Mở đầu

1.1 ĐẶT VẤN ĐỀ

Định vị lỗi là bước tốn công nhất của gỡ lỗi: khi một bộ kiểm thử báo thất bại, lập trình viên phải tìm ra dòng mã gây lỗi trong hàng trăm dòng có liên quan. Họ phương pháp được dùng rộng rãi nhất là định vị lỗi dựa trên phổ độ phủ (SBFL) [4, 5, 6]: một dòng càng hay được các kiểm thử thất bại chạy qua và càng ít được các kiểm thử thành công chạy qua thì càng đáng nghi. Cách làm này rẻ, không cần hiểu ngữ nghĩa chương trình, và hoạt động tốt khi lỗi làm thay đổi đường thực thi.

Với *lỗi ngữ nghĩa* — chương trình chạy đúng đường nhưng tính sai giá trị, ví dụ một phép so sánh lệch biên hay một hằng số trừ thừa một đơn vị — kiểm thử thành công và thất bại thường chạy qua cùng một tập dòng. Phổ độ phủ khi đó gần như không phân biệt được dòng lỗi với các dòng xung quanh. Trên bộ dữ liệu SemFault-250, SBFL theo công thức Ochiai chỉ đặt đúng dòng lỗi ở hạng 1 trong 4,0% số chương trình khi được chấm công bằng (Chương 5).

Mô hình ngôn ngữ lớn (LLM) mở ra hướng ngược lại: đọc mã nguồn, hiểu ý đồ và đoán dòng sai. Các hệ thống định vị lỗi dựa trên LLM như AutoFL [7], AgentFL [8] hay FlexFL [9] cho kết quả ấn tượng, nhưng câu trả lời của LLM mang tính ngẫu nhiên, khó kiểm chứng và có thể chịu ảnh hưởng của việc mô hình đã thấy dữ liệu đánh giá khi huấn luyện [10, 11].

SemLoc [1] — bài báo trung tâm trong ba bài được giảng viên giao — đề xuất một cách dung hòa: bắt LLM viết ra các *ràng buộc ngữ nghĩa có thể thực thi*, chèn chúng vào chương trình, chạy kiểm thử, rồi dùng phổ vi phạm ràng buộc để xếp hạng dòng. Ý tưởng cốt lõi là *neo lập luận của LLM vào bằng chứng thực thi* (structured grounding): LLM đưa giả thuyết, còn kết quả chạy chương trình mới quyết định. SemLoc báo cáo Acc@1 42,8% trên SemFault-250, vượt xa SBFL.

1.2 CÂU HỎI NGHIÊN CỨU

Tiểu luận đặt một câu hỏi đơn giản cho SemLoc: **độ chính xác của nó đến từ bằng chứng thực thi ("grounded") hay từ việc LLM đoán ("guessed")?** Câu hỏi này được tách thành ba câu hỏi nghiên cứu:

- **RQ1 – Tái hiện và kiểm tra cách đánh giá.** Kết quả công bố của SemLoc có tái hiện được không, và còn đứng vững khi chấm điểm công bằng, khi bỏ các thành phần không được mô tả trong bài báo, khi sửa các dòng lỗi chuẩn (ground truth) sai của bộ dữ liệu?
- **RQ2 – Cải tiến và đối chứng.** Bổ sung tín hiệu độ phủ vào SemLoc (cải tiến CovGate) có làm tốt hơn không, và một prompt duy nhất chứa bằng chứng độ phủ (LLM + độ phủ) so với toàn bộ quy trình nhiều bước của SemLoc thì sao, về cả độ chính xác lẫn chi phí?
- **RQ3 – Độ bền.** Các kết luận trên có giữ được trên lỗi thật (BugsInPy [12]), trên mã đã đổi tên định danh (kiểm tra việc LLM ghi nhớ bộ dữ liệu) và trên năm họ mô hình khác nhau không?

1.3 ĐÓNG GÓP CHÍNH

Tiểu luận có các đóng góp sau; mỗi đóng góp gắn với số liệu được sinh tự động từ mã nguồn công khai.

1. **Đo phần "grounded" và phần "guessed".** Mã nguồn công bố cộng cho dòng LLM tự đoán một điểm thưởng lớn hơn mọi điểm ràng buộc, nên dòng đầu tiên của SemLoc *luôn* là dòng LLM đoán: Acc@1 của SemLoc (52,3%) chính là của dòng đoán, còn riêng phổ vi phạm ràng buộc chỉ đạt 43,2%. Bằng chứng thực thi phát huy ở top-3: nó nâng Acc@3 từ 52,3% (chỉ dòng đoán) lên 75,4%.
2. **Một prompt có bằng chứng thực thi vượt quy trình nhiều bước trên các lỗi**

được chạy tới. Phương pháp đối chứng LLM + độ phủ do nhóm thiết kế (một lượt gọi, có tên kiểm thử thất bại, thông báo lỗi và độ phủ từng dòng) hơn SemLoc +16,2 điểm Acc@3 trên các lỗi mà kiểm thử thất bại chạy tới (bộ dòng lỗi chuẩn gốc), tốn ít token hơn khoảng năm lần và không thay đổi đáng kể khi đổi tên định danh; trên toàn bộ nhãn gốc, gồm cả lỗi không kiểm thử nào chạy tới, mức hơn chỉ còn +7,8 điểm và SemLoc tốt hơn ở Acc@5.

3. **Kiểm tra cách đánh giá.** Tính lại khớp toàn bộ cấu hình từ nhật ký công bố, nhưng chỉ ra điểm thưởng ẩn cho dòng LLM đoán, bước kiểm chứng phản thực tế không làm thay đổi thứ hạng nào, mã chấm điểm cho gỡ lỗi chia đôi 38,0% trong khi bài báo ghi 0, hai lỗi trong bộ chạy BugsInPy, và chạy lại với chính mô hình của bài báo không đạt con số trong nhật ký.
4. **SemFault-250 đã sửa.** Dòng lỗi chuẩn được kiểm chứng lại bằng thực thi: 82 được xác nhận, 46 được sửa vị trí, 111 chưa kiểm chứng được và giữ nhãn gốc; 56 dòng chuẩn gốc là dòng trống.
5. **CovGate**, một bước phá hòa bằng Ochiai cho SemLoc với mức tăng nhỏ ở top-3, cùng bảng kiểm tra đánh giá bảy mục áp dụng cho mọi phương pháp.

Ngoài ra, phân tích thất bại (Chương 6) cho thấy độ chính xác cấp dòng phụ thuộc mạnh vào việc dòng lỗi chuẩn được gán ở dòng điều kiện hay ở câu lệnh mà điều kiện đó bảo vệ, và Chương 7 trình bày quan điểm khoa học cùng các mối đe dọa đến tính hợp lệ của kết quả.

1.4 CẤU TRÚC BÁO CÁO

Chương 2 tổng quan tài liệu và bộ từ khóa, trong đó phân tích ba bài báo được giao. Chương 3 trình bày cơ sở lý thuyết về SBFL, lỗi ngữ nghĩa, SemLoc và các công cụ thống kê. Chương 4 mô tả câu hỏi nghiên cứu, dữ liệu, các phương pháp so sánh và thiết kế thực nghiệm. Chương 5 trả lời ba câu hỏi nghiên cứu. Chương 6 phân tích

các ca thất bại; Chương 7 tổng hợp đóng góp, triết lý khoa học và các mối đe dọa. Chương 8 kết luận và nêu hướng phát triển. Phụ lục hướng dẫn tái hiện toàn bộ kết quả. Mã nguồn, dữ liệu và nhật ký gọi LLM được công bố tại `https://github.com/doanan293/grounded-or-guessed`.

Chương 2. Tổng quan tài liệu và từ khóa

2.1 PHƯƠNG PHÁP TÌM VÀ CHỌN TÀI LIỆU

2.1.1 Nguồn và tiêu chí chọn

Điểm xuất phát là ba bài báo giảng viên giao cho chương 10 “Phát hiện lỗi và định vị lỗi”: SemLoc [1], EffiHolmes [2] và báo cáo công nghiệp về khai thác báo cáo sự cố kết hợp LLM [3]. Từ ba bài này, nhóm mở rộng theo hai chiều: lần ngược các tài liệu được trích dẫn (định vị lỗi dựa trên phổ, dựa trên đột biến, gỡ lỗi chia đôi, các bộ dữ liệu lỗi) và tìm xuôi các nghiên cứu mới về định vị lỗi bằng LLM và về tính hợp lệ khi đánh giá LLM. Nguồn tra cứu là arXiv, ACM Digital Library, IEEE Xplore và Google Scholar; mỗi tài liệu được đối chiếu lại qua DOI hoặc mã arXiv trước khi đưa vào danh mục tham khảo.

Một tài liệu được chọn khi thỏa ít nhất một tiêu chí: (i) là phương pháp mà SemLoc so sánh hoặc kế thừa; (ii) là bộ dữ liệu lỗi dùng được cho Python; (iii) bàn về cách đánh giá định vị lỗi (hòa điểm, lỗi thiếu mã, lỗi nhân tạo so với lỗi thật); (iv) bàn về rủi ro khi dùng LLM trong nghiên cứu phần mềm (nhiễm dữ liệu, tái hiện). Kết quả là ba bài được giao và hai mươi tài liệu bổ sung, tổng hợp ở Bảng 2.2.

2.1.2 Bộ từ khóa

Bảng 2.1 là bộ từ khóa song ngữ dùng để tìm kiếm và thống nhất thuật ngữ trong báo cáo.

Bảng 2.1: Bộ từ khóa của đề tài

Tiếng Anh	Tiếng Việt
fault localization	định vị lỗi
spectrum-based fault localization (SBFL)	định vị lỗi dựa trên phổ độ phủ
semantic fault	lỗi ngữ nghĩa
semantic constraint / invariant	ràng buộc ngữ nghĩa / bất biến
semantic violation spectrum	phổ vi phạm ràng buộc ngữ nghĩa
counterfactual verification	kiểm chứng phản thực tế
large language model (LLM)	mô hình ngôn ngữ lớn
structured grounding	neo lập luận vào biểu diễn có cấu trúc
program instrumentation	chèn mã kiểm tra vào chương trình
reproducibility	tính tái hiện được
data contamination / memorization	nhiễm dữ liệu huấn luyện / ghi nhớ
omission fault	lỗi thiếu mã

2.2 BA BÀI BÁO ĐƯỢC GIAO

2.2.1 SemLoc: neo lập luận của LLM vào ràng buộc thực thi được

Vấn đề. Với lỗi ngữ nghĩa, kiểm thử thành công và thất bại chạy qua cùng các câu lệnh, nên phổ độ phủ "sụp" và SBFL mất tín hiệu; còn LLM tự do thì ngẫu nhiên, không kiểm chứng được và khó tách nguyên nhân gốc với triệu chứng.

Phương pháp. SemLoc [1] gồm bốn bước: (1) tìm các *điểm neo* có cấu trúc trong chương trình (đầu hàm, điểm định nghĩa – sử dụng, biên vòng lặp, lệnh trả về); (2) một lượt gọi LLM sinh các ràng buộc ngữ nghĩa theo một ngôn ngữ trung gian khép kín, mỗi ràng buộc gắn với một điểm neo và một biểu thức kiểm tra; (3) chèn các ràng buộc vào chương trình, chạy bộ kiểm thử, dựng ma trận "ràng buộc $\times$ kiểm thử" và chấm từng ràng buộc bằng công thức Ochiai, rồi truyền điểm về câu lệnh thiết lập tính chất đó; (4)

kiểm chứng phản thực tế: với mỗi ràng buộc còn lại, LLM viết một bản vá một dòng tại dòng được quy cho ràng buộc, chạy lại kiểm thử và phân loại ràng buộc theo kết quả.

Đánh giá. Bộ dữ liệu SemFault-250 gồm 250 chương trình Python một hàm, một lỗi, mỗi chương trình có khoảng mười kiểm thử. SemLoc được so với SBFL (Ochiai, Tarantula), gỡ lỗi chia đôi và LLM hỏi thẳng, dùng Claude Sonnet 4.6 và Gemini 2.5 Pro ở ba mức nhiệt độ. Kết quả công bố: Acc@1 42,8%, Acc@3 68,0%; trên lỗi thật, bài báo chỉ có bảng cấp dòng cho bảy lỗi youtube-dl và tự nhận đây là kết quả khả thi.

Điểm cần kiểm chứng. Khi đọc mã nguồn công khai, nhóm thấy: bằng chứng trên lỗi thật rất ít; chương trình của SemFault-250 có văn phong đồng nhất như mã do LLM viết, nên khả năng LLM đã "thấy" dữ liệu chưa được kiểm tra; SemLoc bỏ hẳn tín hiệu độ phủ; mã chấm điểm phá hòa theo thứ tự dòng, tính mọi lần trúng của gỡ lỗi chia đôi là hạng 1 và loại các chương trình chạy lỗi khỏi mẫu số; mỗi cấu hình chỉ chạy một lần, không có kiểm định thống kê. Đây là đầu vào trực tiếp cho ba câu hỏi nghiên cứu.

2.2.2 EffiHolmes: định vị chỗ sửa lỗi hiệu năng bằng so sánh hai lần chạy

EffiHolmes [2] giải bài toán khó hơn ở một khía cạnh: lỗi hiệu năng không làm kiểm thử thất bại và không có vết ngăn xếp, nên các kỹ thuật định vị lỗi kinh điển không áp dụng được. Phương pháp chạy chương trình dưới hai mức tải (mặc định và phóng to), coi các hàm có chi phí tăng theo tải là điểm nóng, trích đường gọi gọn từ điểm vào tới điểm nóng, rồi để LLM lập luận có định hướng trên đường gọi đó. Trên 140 vấn đề hiệu năng thật từ năm kho mã khoa học dữ liệu, EffiHolmes vượt các phương pháp dựa trên truy hồi, tác tử và phân tích hiệu năng đơn thuần, và bền khi dùng mô hình nhỏ.

Bài học cho đề tài: cùng một ý tưởng với SemLoc — *đối chiếu hai lần thực thi để lọc nhiễu trước khi hỏi LLM* — nhưng tín hiệu đối chiếu là chi phí theo tải thay vì kiểm thử đạt/không đạt. Cấu trúc câu hỏi nghiên cứu của bài (hiệu quả, loại bỏ thành phần, độ bền theo cỡ mô hình, phân tích thất bại) được nhóm dùng làm khuôn cho ba câu hỏi của mình. Lỗi hiệu năng nằm ngoài phạm vi thực nghiệm.

2.2.3 Khai thác báo cáo sự cố kết hợp LLM trong công nghiệp

Medeiros và cộng sự [3] khảo sát hai hệ thống Java lớn trong công nghiệp nhận hàng nghìn báo cáo sự cố mỗi tuần. Họ gom cụm vết ngăn xếp, xếp hạng tệp và phương thức đáng nghi, rồi đưa các xếp hạng này vào prompt của LLM; đánh giá hồi cứu năm LLM với bốn cấu hình prompt trên 38 lỗi sự cố thật, có kiểm tra thủ công. Cấu hình tốt nhất định vị được tới 71 trên 100 lỗi và sửa đúng 52 trên 100; thêm bảng xếp hạng phương thức làm tăng tỉ lệ câu trả lời nhắc tới phương thức đã được sửa.

Bài học cho đề tài: (i) *đưa bằng chứng chương trình đã khai thác (xếp hạng) vào prompt* giúp LLM định vị tốt hơn — đây chính là ý tưởng của phương pháp đối chứng LLM + độ phủ mà nhóm xây dựng; (ii) đầu ra của LLM cần được kiểm tra vì hay bị cắt cụt, lệch định dạng hoặc khẳng định không kiểm chứng. Bài cần dữ liệu công nghiệp nên không được dùng để thực nghiệm.

2.3 CÁC HƯỚNG NGHIÊN CỨU LIÊN QUAN

2.3.1 Định vị lỗi dựa trên phổ và đột biến

SBFL bắt đầu với Tarantula [4]; Ochiai [5] và DStar [13] là các công thức mạnh hơn được dùng làm mốc so sánh trong hầu hết nghiên cứu, và là nền của phổ vi phạm trong SemLoc. Các khảo sát của Wong và cộng sự [6] và de Souza và cộng sự [14] chỉ ra hai điểm yếu kinh điển của đánh giá SBFL: xử lý hòa điểm và lỗi thiếu mã (không có dòng nào để đổ lỗi). Định vị lỗi dựa trên đột biến (Metallaxis [15], MUSE [16]) thay độ phủ bằng hành vi của các đột biến; gỡ lỗi chia đôi [17] và chuỗi nhân quả [18] là nền khái niệm cho bước kiểm chứng phản thực tế của SemLoc. Daikon [19] suy ra bất biến từ vết thực thi — SemLoc thay vào đó để LLM đề xuất bất biến.

2.3.2 Định vị lỗi bằng mô hình ngôn ngữ lớn

AutoFL [7] cho LLM gọi công cụ để duyệt mã và giải thích, ở cấp phương thức; SemLoc dùng AutoFL để chọn hàm trước khi xếp hạng dòng trên BugsInPy. AgentFL [8] mở rộng lên cấp dự án bằng nhiều tác tử. FlexFL [9] gần với ý tưởng của nhóm nhất: đưa kết quả SBFL làm gợi ý cho LLM mã nguồn mở, nhưng ở cấp phương thức trên mã Java. Điểm chung là bằng chứng thực thi được đưa vào dưới dạng văn bản; chưa nghiên cứu nào đo riêng phần độ chính xác đến từ bằng chứng và phần đến từ việc LLM tự đoán.

2.3.3 Bộ dữ liệu lỗi và tính hợp lệ của đánh giá

Defects4J [20] (Java) và BugsInPy [12] (Python) cung cấp lỗi thật kèm kiểm thử làm lộ lỗi; QuixBugs [21] gồm các chương trình nhỏ có lỗi một dòng. Pearson và cộng sự [22] cho thấy kết quả định vị lỗi trên lỗi nhân tạo không chuyển được sang lỗi thật, và cách chấm khi hòa điểm hay khi lỗi là thiếu mã có thể đảo thứ hạng giữa các kỹ thuật — đây là cơ sở cho phần kiểm tra cách đánh giá ở RQ1.

2.3.4 Nhiễm dữ liệu và tái hiện khi dùng LLM

Sallou và cộng sự [10] liệt kê các mối đe dọa khi dùng LLM trong nghiên cứu kỹ nghệ phần mềm: mô hình đóng thay đổi theo thời gian, nhiễm dữ liệu, khó tái hiện. Ramos và cộng sự [11] cho thấy LLM có dấu hiệu ghi nhớ các bộ dữ liệu lỗi phổ biến; khảo sát của Xu và cộng sự [23] đề xuất viết lại hoặc đổi tên để giảm nhiễm. Nhóm áp dụng trực tiếp: biến thể đổi tên định danh của SemFault-250 (RQ3), ghi nhật ký và lưu đệm mọi lượt gọi LLM để chạy lại được.

2.4 SO SÁNH VÀ KHOẢNG TRỐNG NGHIÊN CỨU

2.4.1 Bảng so sánh

Bảng 2.2 đặt ba bài được giao cạnh hướng tiếp cận của nhóm.

Bảng 2.2: So sánh ba bài báo được giao với hướng của nhóm

Tiêu chí	SemLoc [1]	EffiHolmes [2]	Báo cáo sự cố + LLM [3]	Nhóm M9
Loại lỗi	ngữ nghĩa	hiệu năng	sự cố (Java)	ngữ nghĩa
Tín hiệu	ràng buộc do LLM sinh + chạy kiểm thử	phân tích hiệu năng ở hai mức tải	vết ngăn xếp + xếp hạng	độ phủ + ràng buộc; đo phần đóng góp của từng tín hiệu
Lỗi thật	7 lỗi có kết quả cấp dòng	140 vấn đề	38 lỗi công nghiệp	31 lỗi BugsInPy, 3 lần chạy
Kiểm tra bộ dữ liệu	không	—	—	kiểm chứng lại dòng lỗi bằng thực thi
Lặp và kiểm định	1 lần, không kiểm định	—	—	3 lần, McNemar, bootstrap theo cụm

2.4.2 Khoảng trống và hướng của nhóm

Ba khoảng trống định hướng toàn bộ tiểu luận. Thứ nhất, các hệ thống định vị lỗi bằng LLM tuyên bố "neo vào thực thi" nhưng chưa ai đo phần độ chính xác thật sự đến từ bằng chứng thực thi. Thứ hai, kết quả được báo cáo từ một lần chạy với cách chấm lạc quan trên một bộ dữ liệu chưa được kiểm chứng. Thứ ba, chưa có so sánh với phương pháp đối chứng đơn giản nhất dùng cùng thông tin: một prompt duy nhất chứa bằng chứng độ

phủ. Từ đó, nhóm học và áp dụng: từ SemLoc — giả thuyết do LLM sinh nhưng kiểm thử phán xử, và dùng chính mã nguồn gốc để tái hiện; từ EffiHolmes — đối chiếu hai lần thực thi để lọc nhiễu (ý tưởng của CovGate) và khuôn câu hỏi nghiên cứu; từ báo cáo công nghiệp — đưa xếp hạng có sẵn vào prompt (phương pháp đối chứng LLM + độ phủ); từ Pearson và cộng sự — kiểm tra cách chấm khi hòa điểm và khi lỗi là thiếu mã.

Chương 3. Cơ sở lý thuyết

3.1 ĐỊNH VỊ LỖI DỰA TRÊN PHỔ ĐỘ PHỦ

3.1.1 Ma trận độ phủ và các công thức nghi ngờ

Cho chương trình P và bộ kiểm thử $T = T_f \cup T_p$ gồm các kiểm thử thất bại T_f và thành công T_p. Với mỗi dòng s, ký hiệu $e_f(s)$, $e_p(s)$ là số kiểm thử thất bại, thành công chạy qua s; $n_f(s)$, $n_p(s)$ là số kiểm thử thất bại, thành công không chạy qua s. Ma trận độ phủ ghi lại, với mỗi cặp (kiểm thử, dòng), kiểm thử đó có chạy qua dòng đó hay không. SBFL gán cho mỗi dòng một điểm nghi ngờ; ba công thức dùng trong tiểu luận là Tarantula [4], Ochiai [5] và DStar [13]:

$$\text{Tarantula}(s) = \frac{e_f/(e_f + n_f)}{e_f/(e_f + n_f) + e_p/(e_p + n_p)}, \tag{3.1}$$

$$\text{Ochiai}(s) = \frac{e_f}{\sqrt{(e_f + n_f)\,(e_f + e_p)}}, \tag{3.2}$$

$$\text{DStar}(s) = \frac{e_f^*}{e_p + n_f}, \quad * = 2. \tag{3.3}$$

Các dòng được xếp giảm dần theo điểm; dòng không được kiểm thử thất bại nào chạy qua ($e_f = 0$) có điểm 0 và trong tiểu luận không được xếp hạng.

3.1.2 Chỉ số đánh giá và cách xử lý hòa điểm

Với mỗi chương trình, *hạng* của lỗi là hạng tốt nhất của một dòng lỗi chuẩn trong danh sách xếp hạng. Acc@k là tỉ lệ chương trình có hạng không vượt quá k:

$$\text{Acc@}k = \frac{1}{|\mathcal{P}|} \sum_{p \in \mathcal{P}} \mathbb{1}[\,\text{rank}_p \le k\,], \tag{3.4}$$

trong đó $\mathcal{P}$ là tập chương trình *cố định*: chương trình mà công cụ không chạy được vẫn nằm trong mẫu số và được tính là trượt. Khi nhiều dòng cùng điểm, có ba cách phá hòa: lấy vị trí đầu (lạc quan, theo thứ tự dòng), vị trí cuối (bi quan) hoặc *hạng trung bình* của nhóm hòa. Tiểu luận dùng hạng trung bình, theo khuyến nghị của các nghiên cứu về đánh giá định vị lỗi [22, 14]; cách chấm của tác giả SemLoc dùng vị trí đầu.

3.2 LỖI NGỮ NGHĨA

3.2.1 Vì sao độ phủ mất tín hiệu

Lỗi ngữ nghĩa là lỗi làm chương trình tính sai giá trị trong khi vẫn chạy qua cùng các câu lệnh như khi đúng — ví dụ `updated = cap - 1` thay vì `updated = cap`. Khi cả kiểm thử thành công và thất bại đều chạy qua dòng lỗi, $e_p(s) > 0$ làm điểm Ochiai của dòng lỗi giảm và ngang với các dòng xung quanh; SBFL khi đó gần như đoán mò trong phạm vi hàm.

3.2.2 Định nghĩa khách quan bằng độ phủ

Để phân tích kết quả theo loại lỗi, tiểu luận dùng một định nghĩa tự động, chỉ dựa trên độ phủ của các dòng lỗi chuẩn:

- **lỗi ngữ nghĩa** (semantic): có dòng lỗi được ít nhất một kiểm thử thất bại *và* một kiểm thử thành công chạy qua — độ phủ không phân biệt được;
- **lộ qua độ phủ** (coverage-revealed): kiểm thử thất bại chạy tới dòng lỗi nhưng không kiểm thử thành công nào chạy tới;
- **không thực thi** (not executed): không kiểm thử thất bại nào chạy tới dòng lỗi chuẩn — thường do dòng lỗi chuẩn bị đặt sai hoặc là dòng trống.

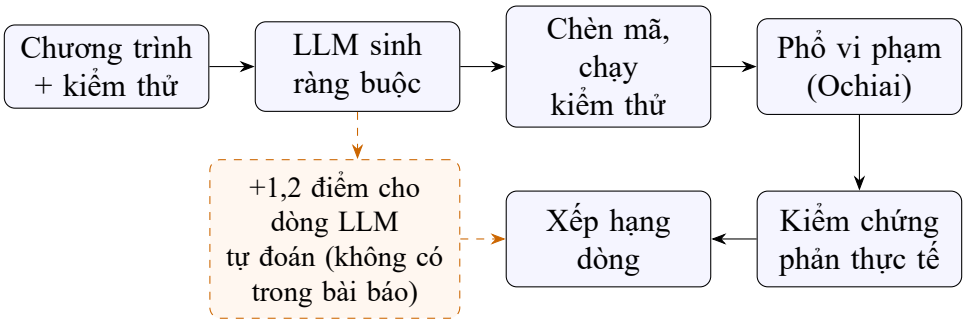
Nhãn được tính lại theo đúng bộ dòng lỗi chuẩn đang dùng để chấm (gốc hoặc đã kiểm

chứng).

3.3 SEMLOC: PHỔ VI PHẠM RÀNG BUỘC NGỮ NGHĨA

3.3.1 Quy trình bốn bước

Hình 3.1 tóm tắt quy trình của SemLoc [1]. Mỗi ràng buộc c do LLM sinh ra là một bộ $\langle$mã, loại, vùng, điểm neo, biểu thức, ý đồ$\rangle$; biểu thức được chèn vào chương trình dưới dạng một lời gọi kiểm tra tại điểm neo. Chạy bộ kiểm thử cho ma trận "ràng buộc $\times$ kiểm thử" ghi ràng buộc nào bị vi phạm trong kiểm thử nào; thay "dòng" bằng "ràng buộc" trong công thức (3.2) cho điểm của từng ràng buộc, và điểm của một dòng là điểm lớn nhất trong các ràng buộc được quy về dòng đó. Bước kiểm chứng phản thực tế loại ràng buộc bị vi phạm cả trong kiểm thử thành công, rồi cho LLM vá thử tại dòng được quy cho từng ràng buộc còn lại và chạy lại kiểm thử.



Hình 3.1: Quy trình của SemLoc (theo [1]); khung nét đứt là thành phần nhóm phát hiện trong mã nguồn

3.3.2 Điểm thưởng cho dòng LLM tự đoán

Khi đọc mã nguồn công khai, nhóm thấy một thành phần không được mô tả trong bài báo: ngoài các ràng buộc, LLM còn trả về một dòng lỗi mà nó tự đoán (trường `fault_line`), và dòng này được cộng thêm 1,2 điểm — lớn hơn điểm tối đa 1 của một ràng buộc. Vì vậy, dòng LLM tự đoán *luôn* đứng hạng 1, bất kể phổ vi phạm ra sao. Thành phần này

là trọng tâm của câu hỏi "grounded hay guessed" ở các chương sau.

3.4 CÔNG CỤ THỐNG KÊ

3.4.1 Kiểm định cặp

Mọi so sánh giữa hai phương pháp đều là so sánh *cặp* trên cùng các chương trình. Với chỉ số nhị phân (trúng hay trượt top-k), kiểm định McNemar dùng số chương trình chỉ phương pháp A trúng (b) và chỉ phương pháp B trúng (c), bản chính xác khi $b + c$ nhỏ. Với hạng nghịch đảo ($1/\text{rank}$, trượt là 0), kiểm định dấu hạng Wilcoxon cùng hệ số tương quan hạng hai chuỗi ghép cặp làm độ lớn hiệu ứng:

$$r_{rb} = \frac{R^+ - R^-}{R^+ + R^-}, \tag{3.5}$$

với R^+, R^- là tổng hạng của các chênh lệch dương, âm (bỏ chênh lệch bằng 0).

3.4.2 Khoảng tin cậy bootstrap theo cụm và giá trị Shapley

Mỗi cấu hình được chạy ba lần. Kết quả của một chương trình được lấy trung bình qua ba lần *trước khi* lấy mẫu lại, nên đơn vị lấy mẫu là chương trình chứ không phải lần chạy. Vì nhiều chương trình của SemFault-250 sinh ra từ cùng một khuôn mẫu, khoảng tin cậy 95% được tính bằng bootstrap *theo cụm khuôn mẫu* (lấy lại cả cụm), với 10000 lần lấy mẫu và hạt giống cố định.

Để đo phần đóng góp của từng tín hiệu, tiểu luận dùng giá trị Shapley. Với tập tín hiệu N và hàm giá trị $v(S)$ là Acc@k khi chỉ bật các tín hiệu trong S,

$$\phi_i = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!\,(|N| - |S| - 1)!}{|N|!} \Big(v(S \cup \{i\}) - v(S) \Big), \tag{3.6}$$

và $\sum_i \phi_i = v(N)$. Vì Acc@k là trung bình các giá trị trúng/trượt theo chương trình, giá

trị Shapley tổng bằng trung bình giá trị Shapley của từng chương trình; tính chất này cho phép tính khoảng tin cậy bootstrap cho từng ϕ_i.

Chương 4. Phương pháp và thiết kế thực nghiệm

4.1 CÂU HỎI NGHIÊN CỨU VÀ GIẢ THUYẾT

4.1.1 Ba câu hỏi nghiên cứu

RQ1 (tái hiện và kiểm tra cách đánh giá): các con số công bố của SemLoc có tái hiện được từ dữ liệu và mã nguồn của tác giả không; chúng thay đổi thế nào khi (a) chấm điểm công bằng, (b) bỏ các thành phần không được mô tả trong bài báo, (c) dùng dòng lỗi chuẩn đã được kiểm chứng bằng thực thi, (d) chạy lại với mô hình khác?

RQ2 (cải tiến và đối chứng): trên cùng các chương trình và cùng mô hình, (a) cải tiến CovGate có làm SemLoc tốt hơn không và thành phần nào của nó có tác dụng; (b) một prompt duy nhất có bằng chứng độ phủ (LLM + độ phủ) so với toàn bộ quy trình nhiều bước SemLoc thế nào về độ chính xác và chi phí; (c) bao nhiêu phần độ chính xác của SemLoc đến từ bằng chứng thực thi, bao nhiêu từ dòng LLM tự đoán?

RQ3 (độ bền): các kết luận của RQ2 có còn đúng (a) trên lỗi thật của BugsInPy, (b) khi đổi tên mọi định danh và xóa chú thích của chương trình (kiểm tra ghi nhớ), (c) trên năm mô hình thuộc các họ khác nhau?

4.1.2 Giả thuyết và tiêu chí bác bỏ

Giả thuyết chính của tiểu luận là: *ở hạng 1, độ chính xác của SemLoc chủ yếu đến từ dòng LLM tự đoán; bằng chứng thực thi đóng góp chủ yếu ở top-3*. Giả thuyết bị bác bỏ nếu riêng phổ vi phạm đạt Acc@1 không thấp hơn riêng dòng đoán, hoặc nếu ở top-3 phổ không làm tăng độ chính xác so với riêng dòng đoán. Giả thuyết phụ: *một prompt có bằng chứng độ phủ không kém quy trình nhiều bước*; bị bác bỏ nếu khoảng tin cậy của chênh lệch Acc@3 nằm hẳn dưới 0.

4.2 DỮ LIỆU

4.2.1 SemFault-250 và cách chia tập

SemFault-250 [1] gồm 250 chương trình Python một hàm, mỗi chương trình có đúng một lỗi được cài vào và khoảng mười kiểm thử. Nhóm chia cố định 50 chương trình làm tập phát triển (dev) — chỉ dùng để chọn tham số — và 200 chương trình làm tập kiểm tra (test) để báo cáo, phân tầng theo nhãn lỗi (mục 3.2) với hạt giống cố định. Vì nhiều chương trình được sinh từ cùng một khuôn mẫu, nhóm gom các chương trình có cấu trúc giống nhau thành cụm (độ tương đồng 4-gram Jaccard từ bốn phần năm trở lên, trên dạng chuẩn hóa của mã) để dùng trong bootstrap theo cụm.

4.2.2 SemFault-250 đã sửa: kiểm chứng dòng lỗi chuẩn bằng thực thi

Kiểm tra dòng lỗi chuẩn cho thấy 56 dòng là dòng trống và 65 chương trình không có kiểm thử thất bại nào chạy tới dòng lỗi chuẩn. Nhóm xây một *bộ phán định sửa tối thiểu*: một dòng được coi là vị trí lỗi nếu thay nó bằng một dòng khác (hoặc chèn một dòng vào dòng trống, với lỗi thiếu mã) với *thay đổi nhỏ* làm *mọi* kiểm thử đạt. LLM chỉ đề xuất các bản sửa ứng viên (GPT-6 luna, hai rồi bốn lần thử ở $T = 0,8$; lượt thứ hai với Claude Sonnet 5.5 cho các dòng còn chưa kiểm chứng); kiểm thử quyết định. Bản sửa được coi là nhỏ khi khoảng cách chỉnh sửa theo từ tố (định danh, số, từng ký tự dấu) so với dòng gốc tối đa là hai hoặc tối đa một phần ba số từ tố của dòng. Các dòng lân cận (trong phạm vi bốn dòng) chỉ được tìm khi không kiểm thử thất bại nào chạy tới dòng chuẩn gốc; vì vậy bộ phán định không phát hiện được trường hợp hai dòng cùng sửa được lỗi. Kết quả: 82 chương trình được *xác nhận* (sửa được ngay tại dòng chuẩn), 46 được *sửa vị trí* (dòng chuẩn không chạy tới được nhưng một dòng gần đó sửa được), 111 *chưa kiểm chứng được* (dòng chuẩn chạy tới được nhưng không bản sửa đề xuất nào đạt) và 11 *không kiểm chứng được* (bị loại). Vị trí "sửa" nằm trên dòng trống không phải câu lệnh nào để xếp hạng nên bị loại; bộ dòng lỗi chuẩn đã sửa còn 226 chương trình. Mọi

kết quả được báo cáo trên cả bộ gốc và bộ đã sửa.

4.2.3 Lỗi thật BugsInPy

Nhóm dựng môi trường cho các lỗi của dự án youtube-dl trong BugsInPy [12] và giữ những lỗi thỏa đủ bốn điều kiện: môi trường cài được, kiểm thử của lỗi tái hiện được thất bại trên phiên bản lỗi, đạt trên phiên bản đã sửa, và đo được độ phủ — được 31 lỗi. Hàm đích là hàm chứa dòng bị lập trình viên sửa mà kiểm thử thất bại được khai báo thực sự chạy qua; dòng lỗi chuẩn là các dòng bị sửa trong hàm đó. Kiểm thử thất bại do môi trường (không được BugsInPy khai báo) bị loại khỏi ma trận độ phủ.

4.2.4 Biến thể đổi tên định danh

Để kiểm tra khả năng mô hình ghi nhớ SemFault-250 [11, 23], nhóm đổi tên mọi định danh do chương trình tự định nghĩa (hàm, tham số, biến, hằng) thành tên vô nghĩa, xóa chú thích và docstring nhưng *giữ nguyên số dòng*, chỉ giữ các tên mà kiểm thử phụ thuộc. Mỗi biến thể được kiểm tra tự động: cùng số dòng, cùng kết quả đạt/không đạt của từng kiểm thử như bản gốc; chương trình nào lệch bị loại. Kết quả là 199 chương trình của tập test.

4.3 CÁC PHƯƠNG PHÁP SO SÁNH

4.3.1 Các phương pháp có sẵn

- **SBFL Ochiai, DStar** trên ma trận độ phủ do nhóm thu thập (mỗi kiểm thử một ngữ cảnh độ phủ).
- **LLM hỏi thẳng**: chế độ dự đoán trực tiếp của SemLoc — một lượt gọi chỉ có mã nguồn chương trình, LLM trả về các dòng nghi ngờ; không có thông tin kiểm thử

nào.

- **SemLoc** như phát hành, chạy qua mã nguồn gốc của tác giả, chỉ thay lớp gọi LLM bằng cổng của nhóm (cùng prompt, cùng logic).

4.3.2 CovGate: bổ sung độ phủ vào SemLoc

CovGate lấy điểm dòng của SemLoc và bổ sung bằng chứng độ phủ, gồm bốn thành phần có thể bật/tắt: (1) *cổng độ phủ* — bỏ các dòng không kiểm thử thất bại nào chạy qua; (2) *cộng* λ·Ochiai vào điểm của dòng, với $\lambda \in \{0; 0,01; 0,1; 0,5\}$; (3) *xếp lại bằng kết quả kiểm chứng phản thực tế* — đưa các dòng của ràng buộc được bản vá xác nhận lên đầu, điều mà mã gốc tính ra nhưng không dùng; (4) *chỉ thưởng khi có căn cứ* — giữ điểm thưởng cho dòng LLM đoán chỉ khi dòng đó được kiểm thử thất bại chạy qua và có ràng buộc bị vi phạm quy về nó. Tổ hợp tham số được chọn *chỉ trên tập dev*, theo Acc@3 rồi Acc@1, gộp qua năm cấu hình công bố; tập test không bao giờ được dùng để chọn. Cấu hình được chọn không bật cổng độ phủ, giữ nguyên điểm thưởng và dùng $\lambda = 0,01$ (bộ dòng lỗi chuẩn gốc) hoặc $\lambda = 0,5$ (bộ đã sửa); nói cách khác, CovGate được đánh giá là một bước *phá hòa bằng Ochiai* cho SemLoc.

4.3.3 LLM + độ phủ: một prompt duy nhất

Để trả lời "quy trình nhiều bước có cần không", nhóm xây phương pháp đối chứng đơn giản nhất có cùng thông tin: một lượt gọi LLM duy nhất (nhiệt độ $T = 0,8$) với mã nguồn trong đó mỗi dòng được chú thích số kiểm thử thất bại và thành công chạy qua nó, số kiểm thử đạt/không đạt, cùng tên và dòng lỗi đầu tiên của các kiểm thử thất bại; LLM trả về năm dòng nghi ngờ nhất dưới dạng JSON (nội dung prompt ở Phụ lục A). So với LLM hỏi thẳng, prompt này thêm *cả* kết quả kiểm thử, thông báo lỗi *và* độ phủ, nên phép so sánh đo tác dụng của bằng chứng thực thi trong prompt nói chung, chưa tách riêng độ phủ. Ý tưởng đưa bằng chứng đã khai thác vào prompt lấy từ báo cáo công nghiệp [3]

và FlexFL [9]. Prompt được thiết kế trước khi xem kết quả trên tập test và không được tinh chỉnh.

4.4 KIỂM TRA CÁCH ĐÁNH GIÁ VÀ PHÂN RÃ ĐÓNG GÓP

4.4.1 Kiểm tra cách đánh giá của SemLoc

Từ nhật ký công bố của tác giả, nhóm tính lại mọi cấu hình theo hai cách: *cách chấm của tác giả* (phá hòa theo thứ tự dòng, gỡ lỗi chia đôi trúng bất kỳ dòng nào tính là hạng 1, chương trình chạy lỗi bị loại khỏi mẫu số) và *cách chấm công bằng* (hạng trung bình khi hòa, mẫu số cố định, công thức (3.4)). Nhóm cũng loại bỏ lần lượt các thành phần không được mô tả trong bài báo (điểm thưởng dòng đoán) và kiểm tra bước kiểm chứng phản thực tế có làm thay đổi thứ hạng hay không.

4.4.2 Phân rã "grounded hay guessed"

Điểm dòng của SemLoc là tổng của hai tín hiệu: phổ vi phạm ràng buộc (bằng chứng thực thi) và điểm thưởng cho dòng LLM tự đoán. Nhóm định nghĩa trò chơi hai người chơi {dòng đoán, phổ}: $v(S)$ là Acc@k khi chỉ giữ các tín hiệu trong S; khi giữ cả hai, điểm dòng trùng đúng với SemLoc như phát hành. Giá trị Shapley (3.6) chia Acc@k của SemLoc cho hai tín hiệu; một trò chơi mở rộng bốn người chơi thêm độ phủ (Ochiai) và xếp lại bằng kiểm chứng phản thực tế.

4.5 THIẾT LẬP THỰC NGHIỆM

4.5.1 Mô hình và hạ tầng

Mô hình chính là Gemini 3.8 Flash; RQ3 thêm GPT-6 luna, Claude Sonnet 5.5, gpt-oss-120b (mã nguồn mở) và Gemini 3.5 Flash-Lite trên một tập con cố định 50 chương trình

của tập test; Claude Sonnet 4.6 chỉ dùng để đối chiếu với nhật ký của tác giả. Mọi mô hình chạy ở mức suy luận mặc định của nhà cung cấp, qua một cổng tương thích OpenAI; SemLoc chạy ở nhiệt độ $T = 0,8$ như cấu hình chính của bài báo. Mọi lượt gọi LLM được ghi nhật ký (mô hình thực sự trả lời, số token, thời gian, lỗi) và lưu đệm theo nội dung và mã lượt chạy, nên toàn bộ thực nghiệm phát lại được mà không gọi mạng.

4.5.2 Lặp, kiểm tra sức khỏe lượt chạy và thống kê

Mỗi cấu hình có LLM chạy 3 lần độc lập; mỗi lần là một mã lượt chạy và một thư mục làm việc riêng. Một lượt chạy chỉ được dùng khi đã chạy xong và qua kiểm tra sức khỏe: không còn lỗi gọi LLM chưa khắc phục, không có chương trình nào lỗi ở bước kiểm chứng phản thực tế. Kết quả báo cáo là trung bình $\pm$ độ lệch chuẩn qua 3 lần; so sánh cặp dùng chênh lệch Δ kèm khoảng tin cậy bootstrap theo cụm, McNemar cho từng lần chạy và Wilcoxon kèm r_{rb} (mục 3.4). Chi phí được đo bằng số lượt gọi, số token và thời gian trên mỗi chương trình.

Chương 5. Kết quả thực nghiệm

Mọi con số trong chương là phần trăm (Acc@k) hoặc điểm phần trăm (Δ), sinh tự động từ dữ liệu; khoảng trong ngoặc vuông là khoảng tin cậy 95% bootstrap theo cụm khuôn mẫu. Có khoảng hai mươi phép so sánh như vậy và nhóm không hiệu chỉnh đa so sánh, nên các khoảng sát 0 cần được đọc thận trọng.

5.1 RQ1 – TÁI HIỆN VÀ KIỂM TRA CÁCH ĐÁNH GIÁ

5.1.1 Tái hiện và hai cách chấm điểm

Tính lại từ nhật ký công bố khớp từng con số của cả 13 cấu hình trong bảng kết quả của SemLoc. *Chạy lại* SemLoc với chính mô hình của bài báo (Claude Sonnet 4.6) trên tập dev thì không khớp: Acc@1 32,0% và Acc@3 44,0% so với 36,0% và 68,0% trong nhật ký trên cùng các chương trình; ràng buộc sinh ra ít bị vi phạm trong kiểm thử thất bại hơn, và nhóm không tìm thấy nguyên nhân ở hạ tầng của mình.

Bảng 5.1 so sánh cách chấm của tác giả với cách chấm công bằng. Ba điểm nổi bật:

- **Gỡ lỗi chia đôi**: mã chấm điểm công bố cho 38,0% (mọi lần trúng tính là hạng 1), trong khi bài báo ghi 0 và cách chấm công bằng cho 0,0%, vì phương pháp này không xếp hạng dòng nào.
- **SBFL bị đánh giá thấp ở top-5**: Acc@5 của Ochiai là 30,4% theo cách của tác giả và 48,0% khi dùng hạng trung bình.
- **SemLoc** gần như không đổi khi chấm công bằng, nhưng **bỏ điểm thưởng cho dòng LLM tự đoán** — thành phần không có trong bài báo — làm Acc@1 giảm từ 42,8% xuống 30,0% (Claude) và từ 39,6% xuống 27,2% (Gemini).

Bảng 5.1: Kết quả công bố tính lại theo hai cách chấm (nhật ký của tác giả, 250 chương trình)

	Cách chấm của tác giả			Cách chấm công bằng		
Cấu hình (cả 250 chương trình)	Acc@1	Acc@3	Acc@5	Acc@1	Acc@3	Acc@5
SemLoc, Claude Sonnet 4,6, $T{=}0,8$	42,8	68,0	76,8	42,8	65,6	76,8
SemLoc, Gemini 2,5 Pro, $T{=}0,8$	39,8	64,7	73,5	39,6	64,4	72,8
SemLoc Claude $T{=}0,8$ không có điểm thưởng dòng đoán	–	–	–	30,0	55,6	66,4
SemLoc Gemini $T{=}0,8$ không có điểm thưởng dòng đoán	–	–	–	27,2	55,6	64,0
SemLoc không CF (lượt chạy riêng), Claude	30,8	59,6	68,8	–	–	–
LLM hỏi thẳng, Claude	35,2	41,2	41,2	35,2	41,2	41,2
SBFL Ochiai	6,4	13,2	30,4	4,0	16,0	48,0
SBFL DStar (tính lại)	–	–	–	4,0	16,0	48,0
Gỡ lỗi chia đôi (DD)	38,0	38,0	38,0	0,0	0,0	0,0

5.1.2 Kiểm chứng phản thực tế và lỗi của gói mã công bố

Trong mã nguồn công bố, kết quả của bước kiểm chứng phản thực tế được tính ra nhưng *không dùng để xếp hạng*: thứ hạng có và không có bước này trùng nhau ở cả 250 chương trình của cả năm cấu hình có dữ liệu. Con số "không có kiểm chứng phản thực tế" trong bài báo đến từ một lượt chạy riêng không công bố điểm từng chương trình; Acc@1 của nó (30,8%) gần bằng SemLoc khi bỏ điểm thưởng, gợi ý một khác biệt về cấu hình hơn là tác dụng của bước này, nhưng nhóm không xác định được. Bộ chạy BugsInPy của SemLoc có thêm hai lỗi: (i) chỉ số "hạng tốt nhất" nó báo cáo là hạng của *chính dòng LLM đoán*, không phải của dòng lập trình viên đã sửa; (ii) bước vá thử sửa tệp nguồn rồi khôi phục trong cùng một giây với cùng kích thước tệp, để lại bản biên dịch Python (`.pyc`) của phiên bản *đã vá*, khiến lỗi "biến mất" ở lượt chạy sau. Nhóm đã sửa lỗi thứ hai trong bộ chạy của mình và chấm lại BugsInPy theo dòng đã sửa thật.

5.1.3 Ảnh hưởng của dòng lỗi chuẩn

Trên lượt chạy của nhóm, chuyển từ dòng lỗi chuẩn gốc (200 chương trình) sang bộ đã sửa (179 chương trình) làm Acc@1 của SemLoc tăng từ 36,8% lên 52,3%. Một phần của mức tăng này là do mẫu số nhỏ hơn: bộ đã sửa bỏ các lỗi không kiểm chứng được. Chỉ tính các lỗi được kiểm thử thất bại chạy tới trên bộ gốc, SemLoc đã đạt 49,8%. SBFL tăng ít hơn nhiều (Acc@3 từ 15,5% lên 19,0%).

5.2 RQ2 – CẢI TIẾN VÀ ĐỐI CHỨNG

5.2.1 Một prompt và quy trình nhiều bước

Bảng 5.2 là kết quả trên 200 chương trình của tập test, ba lần chạy Gemini 3.8 Flash. Kết quả phụ thuộc vào việc lỗi có được kiểm thử thất bại chạy tới hay không. **Trên các lỗi được chạy tới**, LLM + độ phủ hơn SemLoc ở Acc@3 +16,2 điểm [+9,4; +22,8] trên bộ dòng lỗi chuẩn gốc (148 chương trình) và +16,6 điểm [+10,5; +22,8] trên bộ đã sửa. **Trên toàn bộ nhãn gốc**, gồm cả lỗi không kiểm thử nào chạy tới, mức hơn chỉ còn +7,8 điểm [+0,2; +15,2] và SemLoc tốt hơn ở Acc@5 (75,0% so với 70,0%): một prompt dựa trên độ phủ không thể chỉ vào dòng mà không kiểm thử nào chạy qua. Ở Acc@1, mức hơn (+4,8 [+1,5; +8,7]) chỉ thấy rõ khi gộp ba lần chạy; từng lần riêng chưa đạt ý nghĩa thống kê. **LLM hỏi thẳng** (chỉ có mã nguồn) không kém SemLoc ở Acc@1 (Δ = +4,5, khoảng tin cậy chứa 0) nhưng Acc@3 thay đổi −12,7 điểm. Vì LLM + độ phủ thêm cùng lúc kết quả kiểm thử, thông báo lỗi và độ phủ, khoảng cách này đo tác dụng của bằng chứng thực thi trong prompt nói chung, chưa tách riêng độ phủ.

Bảng 5.2: RQ2: so sánh trên tập test (Gemini 3.8 Flash, 3 lần chạy; Δ so với SemLoc)

Phương pháp	Acc@1	Acc@3	Acc@5	ΔAcc@1 [CI 95%]	ΔAcc@3 [CI 95%]	r_{rb}
Dòng lỗi chuẩn gốc, $n = 200$, 3 lần lặp						
SemLoc (bản phát hành)	36,8±0,8	60,3±2,3	75,0±0,9	–	–	–
bỏ điểm thưởng dòng đoán	29,2±1,4	57,7±2,5	72,7±1,3	-7,7 [-13,9; -1,6]	-2,7 [-4,9; -0,9]	-0,38
chỉ thưởng khi có căn cứ	34,7±1,5	58,2±2,9	72,8±1,5	-2,2 [-4,2; -0,6]	-2,2 [-4,2; -0,6]	-0,94
+ chỉ cổng độ phủ	36,8±0,8	57,7±1,8	62,5±1,3	+0,0 [0,0; 0,0]	-2,7 [-7,7; 1,7]	-0,59
+ chỉ λ·Ochiai	36,8±0,8	61,3±1,9	75,0±0,9	+0,0 [0,0; 0,0]	+1,0 [0,2; 2,3]	+0,54
+ xếp lại bằng CF	35,0±1,0	62,2±2,5	77,7±1,8	-1,8 [-3,8; -0,3]	+1,8 [0,5; 3,5]	+0,36
CovGate (nhóm đề xuất)	36,8±0,8	61,3±1,9	75,0±0,9	+0,0 [0,0; 0,0]	+1,0 [0,2; 2,3]	+0,54
CovGate + xếp lại bằng CF	35,0±1,0	62,5±2,2	77,7±1,8	-1,8 [-3,8; -0,3]	+2,2 [0,9; 3,8]	+0,46
LLM hỏi thẳng (1 prompt)	40,0±0,0	43,0±0,5	43,2±0,8	+3,2 [-0,8; 7,4]	-17,3 [-23,3; -11,1]	-0,51
LLM + độ phủ (1 prompt, nhóm đề xuất)	40,2±0,6	68,2±0,3	70,0±0,9	+3,3 [0,5; 6,5]	+7,8 [0,2; 15,2]	+0,04
SBFL Ochiai	3,0±0,0	15,5±0,0	48,0±0,0	-33,8 [-41,2; -26,8]	-44,8 [-52,8; -36,6]	-0,87
Dòng lỗi chuẩn đã sửa bằng thực thi, $n = 179$, 3 lần lặp						
SemLoc (bản phát hành)	52,3±0,9	75,4±2,0	82,7±1,1	–	–	–
bỏ điểm thưởng dòng đoán	43,2±2,1	70,8±2,3	78,4±1,7	-9,1 [-16,7; -1,9]	-4,7 [-7,9; -2,1]	-0,37
chỉ thưởng khi có căn cứ	48,2±1,7	71,3±2,8	78,6±2,0	-4,1 [-7,2; -1,7]	-4,1 [-7,2; -1,7]	-1,00
+ chỉ cổng độ phủ	52,3±0,9	79,1±2,0	85,1±1,3	+0,0 [0,0; 0,0]	+3,7 [1,5; 6,6]	+1,00
+ chỉ λ·Ochiai	52,3±0,9	79,3±1,7	85,1±1,3	+0,0 [0,0; 0,0]	+3,9 [1,6; 6,8]	+0,85
+ xếp lại bằng CF	49,9±0,9	76,2±1,8	83,4±1,4	-2,4 [-4,8; -0,6]	+0,7 [0,0; 2,1]	-0,35
CovGate (nhóm đề xuất)	52,3±0,9	79,3±1,7	85,1±1,3	+0,0 [0,0; 0,0]	+3,9 [1,6; 6,8]	+0,85
CovGate + xếp lại bằng CF	49,9±0,9	77,1±2,2	85,3±1,2	-2,4 [-4,8; -0,6]	+1,7 [0,5; 3,3]	+0,28
LLM hỏi thẳng (1 prompt)	56,8±0,7	62,8±0,7	62,9±0,9	+4,5 [-0,4; 9,7]	-12,7 [-19,7; -6,2]	-0,28
LLM + độ phủ (1 prompt, nhóm đề xuất)	57,2±0,9	92,0±0,3	93,7±0,9	+4,8 [1,5; 8,7]	+16,6 [10,5; 22,8]	+0,65
SBFL Ochiai	3,9±0,0	19,0±0,0	67,6±0,0	-48,4 [-58,6; -38,3]	-56,4 [-67,0; -45,4]	-0,85

5.2.2 Theo loại lỗi và từng thành phần của CovGate

Bảng 5.3 tách kết quả theo nhãn lỗi của bộ đã sửa. Phần hơn của LLM + độ phủ không chỉ đến từ các lỗi mà độ phủ tự làm lộ: trên lỗi ngữ nghĩa thuần — nơi kiểm thử đạt và không đạt chạy qua cùng dòng — Acc@3 vẫn là 91,8% so với 77,6%.

Bảng 5.3: RQ2 theo nhãn lỗi (dòng lỗi chuẩn đã sửa)

		SBFL		SemLoc		CovGate		LLM + độ phủ	
Nhãn lỗi	n	Acc@1	Acc@3	Acc@1	Acc@3	Acc@1	Acc@3	Acc@1	Acc@3
Ngữ nghĩa	162	1,8	17,3	54,7	77,6	54,7	81,1	60,1	91,8
Lộ qua độ phủ	17	23,5	35,3	29,4	54,9	29,4	62,7	29,4	94,1
Thực thi được (trừ nhóm không thực thi)	179	3,9	19,0	52,3	75,4	52,3	79,3	57,2	92,0

CovGate được chọn trên tập dev là một bước phá hòa bằng Ochiai (mục 4.3); nó thay đổi Acc@3 +3,9 điểm [+1,6; +6,8] trên bộ đã sửa và +1,0 điểm [+0,2; +2,3] trên bộ gốc, không đổi Acc@1. Riêng cổng độ phủ cho mức tăng tương tự trên bộ đã sửa nhưng thay đổi Acc@3 −2,7 điểm trên bộ gốc, vì nó loại mọi lỗi không được chạy tới. "Chỉ thưởng khi có căn cứ" thay đổi Acc@1 −4,1 điểm và xếp lại bằng kiểm chứng phản thực tế thay đổi −2,4 điểm: bằng chứng thực thi không phân biệt được dòng đoán đúng với dòng đoán sai.

5.2.3 Grounded hay guessed: giá trị biên và phân rã

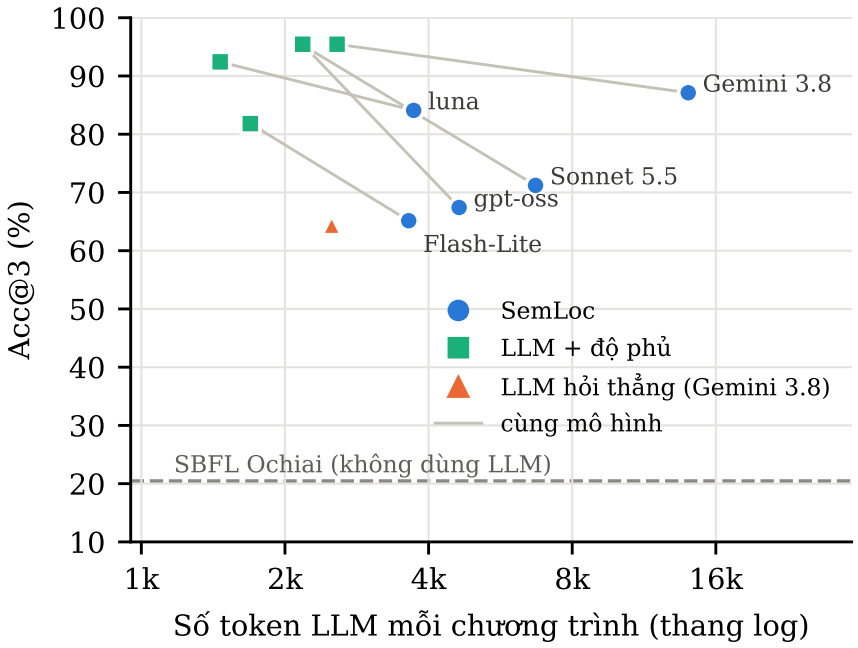
Vì điểm thưởng lớn hơn mọi điểm ràng buộc, thêm phổ vào dòng đoán *không thể* thay đổi dòng đầu tiên: riêng dòng đoán và SemLoc cùng đạt Acc@1 52,3%, còn riêng phổ đạt 43,2%. Giá trị Shapley của phổ ở Acc@1 (Bảng 5.4, 21,6 điểm) vì vậy là phần công cho điều phổ *sẽ* đạt được nếu không có dòng đoán, không phải đóng góp vào dòng đầu của SemLoc. Ở top-3, phổ có tác dụng thật: dòng đoán chỉ nêu một dòng nên dừng ở 52,3%, riêng phổ đạt 70,8%, và cả hai cùng đạt 75,4%. Kết luận: **hạng 1 là "guessed" theo thiết kế, top-3 là nơi bằng chứng thực thi có tác dụng**. Trên nhật ký của tác giả (Claude), phân rã cho 42,8% = 27,8 (dòng đoán) + 15,0 (phổ).

Bảng 5.4: Phân rã Shapley độ chính xác của SemLoc (lượt chạy của nhóm, 3 lần)

Dòng lỗi chuẩn	Chỉ số	Tổng	Dòng LLM đoán	Phổ ngữ nghĩa	Độ phủ	CF
SemLoc như phát hành (dòng đoán + phổ)						
Bản gốc ($n = 200$)	Acc@1	36,8	22,2 [16,5; 28,4]	14,6 [11,8; 17,5]	–	–
Bản gốc ($n = 200$)	Acc@3	60,3	19,8 [16,1; 23,8]	40,6 [35,6; 45,3]	–	–
Đã sửa ($n = 179$)	Acc@1	52,3	30,7 [23,5; 37,8]	21,6 [17,4; 26,1]	–	–
Đã sửa ($n = 179$)	Acc@3	75,4	28,5 [23,4; 33,5]	46,9 [41,9; 51,7]	–	–
Mọi tín hiệu (+ Ochiai thô, + xếp lại bằng CF)						
Bản gốc ($n = 200$)	Acc@1	35,0	22,7 [17,4; 28,3]	12,5 [10,0; 15,2]	-0,5 [-1,5; 0,6]	0,3 [-0,6; 1,1]
Bản gốc ($n = 200$)	Acc@3	59,0	17,6 [14,2; 21,4]	35,8 [31,6; 40,2]	4,6 [0,1; 9,0]	1,1 [0,5; 1,8]
Đã sửa ($n = 179$)	Acc@1	49,9	31,9 [25,3; 38,5]	18,6 [14,4; 22,9]	-1,1 [-2,3; 0,2]	0,5 [-0,7; 1,5]
Đã sửa ($n = 179$)	Acc@3	80,6	25,9 [20,8; 31,0]	44,4 [39,7; 49,3]	9,4 [5,8; 13,4]	0,9 [0,3; 1,7]

5.2.4 Chi phí

SemLoc tốn trung bình 14,0 nghìn token cho mỗi chương trình, trong đó bước kiểm chứng phản thực tế chiếm 42% số lượt gọi LLM mà không cải thiện thứ hạng; LLM + độ phủ chỉ tốn 2,6 nghìn token. Hình 5.1 đặt độ chính xác cạnh chi phí cho năm mô hình trên cùng một tập con (44 chương trình): với mọi mô hình, LLM + độ phủ có Acc@3 cao hơn và tốn ít token hơn SemLoc. Các so sánh theo từng mô hình chưa có khoảng tin cậy; ở Acc@1, GPT-6 luna không hơn, và trên tập con này LLM hỏi thẳng có Acc@1 cao nhất với Gemini 3.8 Flash (59,9% so với 56,1%).



Hình 5.1: Chi phí và độ chính xác (Acc@3) trên cùng tập con, dòng lỗi chuẩn đã sửa; đường nối hai phương pháp của cùng một mô hình

5.3 RQ3 – ĐỘ BỀN

5.3.1 Lỗi thật BugsInPy

Bảng 5.5 cho kết quả trên 31 lỗi của một dự án (youtube-dl), ba lần chạy. Theo thiết kế, dòng đầu của SemLoc là dòng LLM đoán, nên Acc@1 của SemLoc chính là của dòng đoán (46,2%); riêng phổ ngữ nghĩa chỉ đặt đúng 7,5% ở hạng 1, gần SBFL (9,7%). LLM + độ phủ đi cùng chiều với kết quả trên SemFault-250 — Acc@3 69,9% so với 53,8% — nhưng với 31 lỗi, khoảng tin cậy [−3,2; +34,4] vẫn chứa 0. “Lỗi được chạy tới” là các lỗi có kiểm thử thất bại chạy qua một dòng lỗi chuẩn; chỉ số của gói mã công bố (dòng cuối bảng) là hạng của dòng LLM đoán, đo mức nhất quán của LLM với chính nó chứ không đo khả năng định vị.

Bảng 5.5: RQ3a: 31 lỗi thật youtube-dl (BugsInPy), 3 lần chạy

	Mọi lỗi ($n = 31$)			Lỗi được chạy tới ($n = 26$)	
Phương pháp	Acc@1	Acc@3	Acc@5	Acc@1	Acc@3
Chỉ phổ ngữ nghĩa của SemLoc	7,5±1,9	29,0±3,2	37,6±3,7	9,0±2,2	34,6±3,9
SemLoc (phổ + điểm thưởng dòng đoán)	46,2±6,7	53,8±7,4	55,9±6,7	51,3±8,0	60,3±8,9
Chỉ dòng LLM đoán	46,2±6,7	46,2±6,7	46,2±6,7	51,3±8,0	51,3±8,0
CovGate (nhóm đề xuất)	46,2±6,7	54,8±6,5	57,0±7,4	51,3±8,0	61,5±7,7
LLM + độ phủ (1 prompt, nhóm đề xuất)	51,6±5,6	69,9±3,7	75,3±1,9	59,0±5,9	75,6±4,4
SBFL Ochiai	9,7±0,0	29,0±0,0	29,0±0,0	11,5±0,0	34,6±0,0
SBFL DStar	9,7±0,0	29,0±0,0	29,0±0,0	11,5±0,0	34,6±0,0
Chỉ số của gói mã công bố (hạng của dòng LLM đoán)	39,8±1,9	45,2±3,2	51,6±3,2	37,2±2,2	43,6±5,9

5.3.2 Đổi tên định danh

Bảng 5.6 so sánh từng chương trình với bản đổi tên của nó, theo cặp lần chạy (chỉ với Gemini 3.8 Flash). Dòng LLM đoán không thay đổi đáng kể (Acc@1 $\Delta = -2{,}6$ [−7,8; +2,4]), nên không có dấu hiệu mô hình đoán nhờ nhớ tên. Ngược lại, phổ vi phạm ràng buộc cần tên có nghĩa: không có điểm thưởng, Acc@3 thay đổi −15,5 điểm. LLM + độ phủ không thay đổi đáng kể: Acc@3 $\Delta = -0{,}9$ [−3,5; +1,3].

Bảng 5.6: RQ3b: bản gốc và bản đổi tên định danh (dòng lỗi chuẩn đã sửa, 3 cặp lần chạy)

	Acc@1			Acc@3		
Phương pháp	Bản gốc	Đổi tên	Δ [CI 95%]	Bản gốc	Đổi tên	Δ [CI 95%]
SemLoc	52,1±0,9	49,4±1,0	-2,6 [-7,8; 2,4]	75,3±2,0	66,8±2,0	-8,4 [-14,1; -3,0]
bỏ điểm thưởng dòng đoán	42,9±2,1	34,5±2,1	-8,4 [-13,5; -3,7]	70,6±2,3	55,1±2,6	-15,5 [-21,4; -10,3]
Chỉ dòng LLM đoán	52,1±0,9	49,4±1,0	-2,6 [-7,8; 2,4]	52,1±0,9	49,4±1,0	-2,6 [-7,8; 2,4]
LLM + độ phủ (1 prompt)	56,9±0,9	56,2±0,0	-0,8 [-3,6; 2,0]	92,0±0,3	91,0±1,5	-0,9 [-3,5; 1,3]

5.3.3 Các họ mô hình

Trên tập con (Bảng 5.7, chưa có khoảng tin cậy), mô hình yếu hơn dựa vào điểm thưởng nhiều hơn: bỏ điểm thưởng, Acc@1 của Gemini 3.5 Flash-Lite giảm từ 42,4% xuống 20,4%, của gpt-oss-120b từ 45,5% xuống 29,5%. Với Gemini 3.8 Flash, tập con gần như không đổi (52,3% so với 53,0%) trong khi trên toàn tập test Acc@1 thay đổi −9,1 điểm; tập con quá nhỏ để xếp hạng các mô hình theo hiệu ứng này.

Bảng 5.7: RQ3c: năm mô hình trên cùng tập con (dòng lỗi chuẩn đã sửa, 3 lần chạy)

		SemLoc		bỏ điểm thưởng dòng đoán		CovGate
Mô hình	Số lần lặp	Acc@1	Acc@3	Acc@1	Acc@3	Acc@3
Gemini 3,8 Flash	3	52,3±2,3	87,1±1,3	53,0±4,7	84,1±0,0	87,9±2,6
GPT-6 luna	3	53,8±2,6	84,1±2,3	40,9±6,0	72,7±2,3	84,9±1,3
Claude Sonnet 5,5	3	43,2±2,3	71,2±1,3	37,9±1,3	68,9±1,3	71,2±1,3
gpt-oss-120b	3	45,5±6,0	67,4±3,5	29,5±7,9	57,6±6,6	68,2±2,3
Gemini 3,5 Flash-Lite	3	42,4±3,5	65,1±5,7	20,4±2,3	51,5±4,7	71,2±1,3

5.4 TÓM TẮT TRẢ LỜI CÁC CÂU HỎI NGHIÊN CỨU

Bảng 5.8: Trả lời ba câu hỏi nghiên cứu

Câu hỏi	Trả lời
RQ1	Tính lại từ nhật ký khớp toàn bộ, chạy lại thì không; điểm thưởng ẩn quyết định dòng đầu; bước kiểm chứng phản thực tế không đổi thứ hạng; mã chấm điểm lạc quan với gỡ lỗi chia đôi và bi quan với SBFL; lỗi của bộ dữ liệu làm thay đổi đáng kể các con số.
RQ2	Trên lỗi được chạy tới, một prompt có bằng chứng thực thi vượt quy trình nhiều bước ở Acc@3 với chi phí thấp hơn nhiều; trên toàn bộ nhãn gốc mức hơn nhỏ lại và SemLoc tốt hơn ở Acc@5. CovGate giúp ít. Hạng 1 là "guessed" theo thiết kế, top-3 là nơi bằng chứng thực thi có tác dụng.
RQ3	Trên 31 lỗi thật, kết quả cùng chiều nhưng chưa đủ mẫu; đổi tên làm yếu phổ ràng buộc nhưng không làm thay đổi đáng kể dòng đoán hay LLM + độ phủ (một mô hình); mô hình yếu dựa vào dòng đoán nhiều hơn (tập con, chưa có khoảng tin cậy).

Chương 6. Phân tích thất bại và thảo luận

Chương này trả lời câu hỏi: khi SemLoc (hoặc dòng LLM tự đoán) sai, nó sai thế nào và vì sao? Phân tích dùng ba lần chạy Gemini 3.8 Flash trên tập test với bộ dòng lỗi chuẩn đã sửa; tín hiệu nhị phân của mỗi chương trình lấy theo đa số ba lần.

6.1 PHÂN NHÓM TỰ ĐỘNG

6.1.1 Sáu nhóm kết quả

Mỗi chương trình được xếp vào đúng một nhóm theo thứ tự ưu tiên của Bảng 6.1. Trên 179 chương trình, khoảng một nửa được giải quyết bởi chính dòng LLM tự đoán; 25 chương trình là nơi bằng chứng thực thi thật sự "cứu" SemLoc khi dòng đoán sai — đây là phiên bản theo từng chương trình của phân rã Shapley ở mục 5.2.

Bảng 6.1: Phân nhóm kết quả theo chương trình (dòng lỗi chuẩn đã sửa)

Nhóm	Định nghĩa	Số chương trình
`gt_suspect`	dòng lỗi chuẩn chưa kiểm chứng được và mọi phương pháp trượt top-3	6
`guess_right`	dòng LLM tự đoán là dòng lỗi chuẩn	94
`prior_hurts`	phổ xếp dòng lỗi hạng 1 nhưng điểm thưởng đẩy dòng đoán sai lên trên	19
`rescued_by_spectrum`	đoán sai nhưng SemLoc vẫn có dòng lỗi trong top-3	25
`coverage_only_wins`	SemLoc trượt top-3, LLM + độ phủ trúng top-3	32
`all_miss`	còn lại: mọi phương pháp trượt	3

6.1.2 Cách chọn ca để đọc

Trong mỗi nhóm trừ `guess_right`, nhóm chọn tối đa bốn ca, ưu tiên các cụm khuôn mẫu khác nhau, với hạt giống cố định, để mẫu đọc trải rộng chứ không lặp lại một khuôn mẫu. Với mỗi ca, công cụ trích đoạn mã quanh dòng lỗi chuẩn và dòng đoán, dòng đoán của từng lần chạy, năm dòng đầu của SemLoc và của LLM + độ phủ, và ràng buộc có điểm cao nhất.

6.2 MƠ HỒ DÒNG KỀ CẠNH

6.2.1 Dòng đoán lệch có hệ thống xuống dưới

Trên mọi lần đoán của ba lần chạy, dòng đoán trùng dòng lỗi chuẩn 281 lần; lệch xuống một dòng 99 lần và hai dòng 61 lần, nhưng lệch lên một dòng chỉ 28 lần và hai dòng 26 lần. LLM + độ phủ lệch cùng chiều (113 lần xuống một dòng so với 24 lần lên một dòng). Phần lớn các lần "sai" vì vậy không phải chỉ vào một chỗ khác, mà chỉ vào dòng ngay bên dưới.

6.2.2 Quan hệ đầu khối – thân khối và độ nhạy của Acc@1

Phân tích cây cú pháp cho thấy, trong 85 chương trình mà dòng đoán sai, 44 ca có dòng đoán và dòng lỗi chuẩn là *đầu một câu lệnh ghép* (`if`, `for`, `while`…) *và câu lệnh mà nó bảo vệ*. Nếu chấp nhận cả hai dòng của cặp này là đúng, Acc@1 của SemLoc tăng từ 52,3% lên 73,9% (biến thể chặt: chỉ dòng đầu khối và dòng đầu tiên của thân) đến 76,2% (cận trên); của LLM + độ phủ từ 57,2% lên 79,3%–81,6%. Hơn hai mươi điểm Acc@1 phụ thuộc vào quy ước gán nhãn — lớn hơn mọi khác biệt giữa các phương pháp dùng LLM ở Chương 5; thứ tự giữa các phương pháp không đổi.

6.3 NĂM MẪU NGUYÊN NHÂN

6.3.1 Dòng điều kiện và câu lệnh được bảo vệ

Mẫu phổ biến nhất (sáu trong mười chín ca đọc): dòng lỗi chuẩn là dòng điều kiện, còn lỗi cài vào nằm ở câu lệnh nó bảo vệ — hoặc ngược lại. Ví dụ trong chương trình `bump_click`:

```
    if cap is not None and counters[key] > cap:    # dòng lỗi chuẩn
        counters[key] = cap - 1                    # LLM chỉ dòng này
```

Lỗi cài vào là `- 1`; LLM đúng về ngữ nghĩa nhưng sai một dòng so với nhãn. Các ca tương tự: `ledger_balance_rollup` (`elif kind == "debit":` và `balance += amount`), `quota_for_fuel` (`if _INTEGER_ALLOCATION:` và phép cắt `int(x)`), và chiều ngược lại ở `gain_trendline`, `estimate_supply`, nơi dòng lỗi chuẩn nằm trong thân khối còn LLM chỉ chính điều kiện bị đảo `if not clamp:`.

6.3.2 Khối lỗi nhiều dòng

Ba ca có lỗi cài vào là cả một khối `if … merged[-1] == left[-1] == right[-1]: merged.pop()`; xóa bất kỳ dòng nào của khối cũng sửa được lỗi. Nhãn chỉ nêu một dòng, LLM chỉ dòng kia — và chính các ca này bị xếp vào `prior_hurts`. Như vậy, phần lớn trường hợp "điểm thưởng gây hại" thật ra là hai dòng tương đương về mặt sửa lỗi.

6.3.3 Bộ phán định chấp nhận bản sửa bù trừ

Bộ phán định sửa tối thiểu của nhóm (mục 4.2) chỉ yêu cầu mọi kiểm thử đạt, nên đôi khi chấp nhận một bản sửa *bù trừ*. Ở `gradient_clip_budget`, dòng chuẩn gốc là dòng trống và được "sửa" thành dòng tính chuẩn bằng `norm = _compute_global_norm(gradients) - 1`, trong khi lỗi thật là `max_norm / (norm + 1)` ở dòng mà

LLM chỉ cả ba lần. Ở `energy_window_avg`, bộ phán định "xác nhận" bằng cách thay một lời gọi ghi log bằng một phép gán. Ở các khối lỗi nhiều dòng, bộ phán định "xác nhận" bằng cách làm điều kiện luôn sai. Các bản sửa đặt trên dòng trống đã bị loại khỏi bộ dòng lỗi chuẩn đã sửa (13 chương trình); bản sửa bù trừ trên dòng có mã thì chưa phát hiện tự động được.

6.3.4 Nguyên nhân và triệu chứng trên cùng luồng dữ liệu

Bốn ca thuộc mẫu này: dòng lỗi chuẩn là nơi tính một giá trị, LLM chỉ nơi dùng giá trị đó, hoặc ngược lại (`allocate_bounty_cut`: trừ phí trên từng phần hay tính phần chia); hoặc lỗi là *thiếu* một bước chuẩn hóa (`shift_trust`, `shift_risk`) hay thiếu một lệnh lưu (`warm_cache_loader`), không có "dòng sai" duy nhất. Đọc mã không đủ để phân xử; cần bộ phán định thực thi kiểm tra cả hai dòng.

6.3.5 Dòng lỗi chuẩn đáng ngờ

Bốn ca còn lại có dòng lỗi chuẩn khó bảo vệ: `reserve_slot` và `schedule_call` gán lỗi cho dòng khai báo một hàm lồng (`def overlaps(…)`); `file_chunk_stitch` gán cho `data += part` trong khi phép cắt `return data[:-1]` ngay dưới mới là lỗi rõ ràng; `room_reservation` có các lần chạy không thống nhất.

6.4 THẢO LUẬN

6.4.1 Đọc "guessed" cho đúng

Kết luận "hạng 1 chủ yếu đến từ dòng LLM đoán" của Chương 5 vẫn đúng, nhưng phân tích thất bại cho thấy dòng đoán thường *đúng về ngữ nghĩa nhưng lệch một dòng* so với nhãn. Vì vậy không thể suy ra "bỏ điểm thưởng thì tốt hơn": các ca điểm thưởng "gây

hại" gần như toàn là cặp đầu khối – thân khối hoặc khối lỗi nhiều dòng. Điều thật sự thiếu là một cách đánh giá chấp nhận các vị trí sửa tương đương.

6.4.2 Khi nào bằng chứng thực thi có ích

Bằng chứng thực thi có ích rõ ràng ở top-3: phổ vi phạm đưa dòng lỗi vào ba dòng đầu ở 25 chương trình mà dòng đoán sai, và bằng chứng độ phủ đưa thẳng cho LLM còn mạnh hơn (32 chương trình SemLoc trượt top-3 nhưng LLM + độ phủ trúng). Nó không giúp phân biệt một dòng đoán đúng với một dòng đoán sai ở hạng 1 (mục 5.2). Bài học cho thiết kế công cụ: dùng bằng chứng thực thi để *thu hẹp* vùng nghi ngờ và đưa nó thẳng cho LLM, thay vì để LLM đoán rồi cộng điểm thưởng.

Chương 7. Đóng góp, triết lý khoa học và các mối đe dọa

7.1 CÁC ĐÓNG GÓP KHOA HỌC

Mỗi đóng góp dưới đây được trình bày dưới dạng: khẳng định, bằng chứng (tệp dữ liệu trong kho mã), điều kiện sẽ bác bỏ nó, và giới hạn.

7.1.1 Đo phần "grounded" và phần "guessed"

Khẳng định. Ở hạng 1, phần lớn độ chính xác của SemLoc đến từ dòng LLM tự đoán, không từ phổ vi phạm ràng buộc. Trên lượt chạy của nhóm, Acc@1 52,3% = 30,7 (dòng đoán) + 21,6 (phổ); trên nhật ký của tác giả, 42,8% = 27,8 + 15,0; trên lỗi thật, Acc@1 của SemLoc bằng đúng Acc@1 của riêng dòng đoán (46,2%). **Bằng chứng:** `data/rq_decomposition.csv`, `data/rq3_bugsinpy.csv`. **Bác bỏ nếu:** bỏ điểm thưởng mà Acc@1 không giảm; ở đây Acc@1 giảm −9,1 điểm [−16,7; −1,9]. **Giới hạn:** giá trị Shapley phụ thuộc cách định nghĩa "tắt" một tín hiệu; dòng đoán thường lệch một dòng so với nhãn (Chương 6).

7.1.2 Với lỗi được kiểm thử chạy tới, một prompt có bằng chứng thực thi vượt quy trình nhiều bước

Khẳng định. Trên bộ dòng lỗi đã sửa (các lỗi mà một kiểm thử thất bại chạy tới), phương pháp đối chứng LLM + độ phủ có Acc@3 92,0% so với 75,4% của SemLoc (Δ +16,6 [+10,5; +22,8]) và Acc@1 cao hơn (Δ +4,8 [+1,5; +8,7]); vẫn hơn trên lỗi ngữ nghĩa thuần; tốn 2,6 nghìn token mỗi chương trình so với 14,0 nghìn; gần như không đổi khi đổi tên định danh (ΔAcc@3 −0,9). **Bằng chứng:** `data/rq2_ours.csv`, `data/rq3_renamed.csv`, `data/llm_cost.csv`. **Giới hạn:** prompt khác chế độ trực tiếp của SemLoc ở nhiều điểm cùng lúc (độ phủ, kết quả kiểm thử, giả định một dòng

lỗi), nên chênh lệch không quy riêng cho độ phủ; trên toàn bộ nhãn gốc, kể cả lỗi không kiểm thử nào chạy tới, phần hơn nhỏ hơn nhiều và SemLoc tốt hơn ở Acc@5; trên 31 lỗi thật, chênh lệch cùng chiều nhưng khoảng tin cậy còn chứa 0; so sánh theo từng mô hình chưa có khoảng tin cậy; prompt do nhóm thiết kế.

7.1.3 Kết quả công bố phụ thuộc vào cách chấm và thành phần không được mô tả

Khẳng định. Tái hiện khớp toàn bộ cấu hình, nhưng: điểm thưởng ẩn cho dòng đoán chiếm phần lớn Acc@1; gỡ lỗi chia đôi đạt 38,0% theo mã chấm của tác giả và 0,0% khi chấm công bằng; Acc@5 của SBFL bị đánh giá thấp (30,4% so với 48,0%); bước kiểm chứng phản thực tế không đổi thứ hạng nào dù chiếm 42% lượt gọi LLM; bộ chạy BugsInPy đo hạng của dòng đoán thay vì dòng đã sửa và để lại bản biên dịch cũ sau bước vá thử. **Bằng chứng:** `data/rq1_audit.csv`, `data/rq1_cf_identity.json`, `src/flx/bip_run.py`.

7.1.4 SemFault-250 đã sửa

Khẳng định. Bộ dòng lỗi chuẩn được kiểm chứng lại bằng thực thi (82 xác nhận, 46 sửa vị trí); dùng nó làm Acc@1 của SemLoc tăng từ 36,8% lên 52,3%. **Giới hạn:** bộ phán định chấp nhận cả bản sửa bù trừ; các vị trí trên dòng trống đã bị loại, còn bản sửa bù trừ trên dòng có mã chưa được phát hiện tự động.

7.1.5 Mơ hồ dòng kề cạnh của Acc@1 cấp dòng

Khẳng định. Với lỗi trải trên dòng điều kiện và câu lệnh được bảo vệ, Acc@1 đổi hơn hai mươi điểm theo quy ước gán nhãn (SemLoc 52,3% $\rightarrow$ 73,9%); thứ tự giữa các phương pháp giữ nguyên. **Bằng chứng:** `data/failure_tolerance.csv`.

7.1.6 CovGate, bảng kiểm tra đánh giá và gói tái hiện

CovGate tăng Acc@3 +3,9 điểm [+1,6; +6,8] và giữ nguyên Acc@1 — gắn thêm bằng chứng vào quy trình nhiều bước chỉ giúp ít so với đưa thẳng bằng chứng cho LLM; với λ chọn trên tập dev, CovGate thực chất là một bộ phá hòa bằng Ochiai. Bảng kiểm tra bảy mục (phá hòa, mẫu số, dòng lỗi kiểm chứng, loại bỏ thành phần ẩn, lặp và kiểm định, chi phí, so với một prompt có cùng bằng chứng) được áp cho mọi phương pháp. Gói tái hiện gồm Docker chạy không cần mạng, toàn bộ nhật ký và bộ đệm lượt gọi LLM để phát lại, và macro sinh tự động cho mọi con số trong báo cáo.

7.2 TRIẾT LÝ KHOA HỌC

7.2.1 Bằng chứng thực thi là trọng tài

Ở mọi chỗ cần phán xử — dòng nào là lỗi, bản sửa nào đúng — nhóm để kiểm thử quyết định, LLM chỉ sinh ứng viên. Đây cũng chính là câu hỏi đặt cho SemLoc: nếu một phương pháp tuyên bố neo lập luận vào thực thi, phần neo đó phải được đo. Tái hiện không phải là chép lại con số: khớp số công bố là điều kiện cần; điều kiện đủ là hiểu vì sao ra con số đó và kết quả còn đứng vững khi đổi cách chấm hợp lý.

7.2.2 Đo đóng góp, so với phương pháp đối chứng đơn giản, và chấp nhận bị bác bỏ

Một con số Acc@1 không nói nó đến từ đâu; phân rã đóng góp và loại bỏ từng thành phần biến câu hỏi "phương pháp có tốt không" thành "phần nào làm nó tốt". Một quy trình nhiều bước chỉ đáng giá khi thắng phương pháp đối chứng đơn giản nhất có cùng thông tin. Mỗi khẳng định đi kèm điều kiện bác bỏ và khoảng tin cậy; đơn vị lấy mẫu là chương trình (hoặc cụm khuôn mẫu), không coi các lần chạy là mẫu độc lập; kết quả được báo cáo cả khi khoảng tin cậy chứa 0. Cuối cùng, bộ dữ liệu cũng là một giả thuyết:

nhãn do người gán có thể sai hoặc mơ hồ, và chính bộ phán định sửa nhãn cũng có giới hạn cần nêu rõ.

7.3 CÁC MỐI ĐE DỌA ĐẾN TÍNH HỢP LỆ

7.3.1 Tính hợp lệ nội tại và tính hợp lệ của cấu trúc

- **Bộ phán định sửa tối thiểu** chấp nhận bản sửa bù trừ nên bộ dòng lỗi đã sửa còn nhiễu; mọi kết quả cũng được báo cáo trên bộ gốc.
- **Bộ dòng lỗi kiểm chứng có thể thiên về phương pháp dùng độ phủ**, vì bộ phán định cần kiểm thử thất bại chạy tới dòng sửa; so sánh giữa SemLoc, các phương pháp đối chứng dùng LLM và CovGate cùng chiều trên bộ gốc, riêng cổng độ phủ thì không.
- **Mơ hồ dòng kề cạnh**: Acc@1 chặt phụ thuộc quy ước gán nhãn; nhóm báo cáo cả Acc@1 chấp nhận đầu khối/thân khối.
- **Prompt phương pháp đối chứng** do nhóm viết trước khi xem kết quả và không tinh chỉnh; prompt khác có thể cho kết quả khác.

7.3.2 Tính hợp lệ ngoại tại

- **Lỗi thật ít và chỉ một dự án** (31 lỗi youtube-dl); hàm đích được chọn từ bản sửa của lập trình viên nên định vị cấp hàm không được đánh giá; một lỗi còn hai kiểm thử thất bại do môi trường trong bộ chạy gốc của SemLoc.
- **Mô hình**: mô hình chính mới hơn mô hình trong bài gốc; mỗi nhà cung cấp chạy ở mức suy luận mặc định, nên so sánh giữa họ mô hình là "ở mặc định" chứ không cùng mức suy luận; Claude qua cổng trung gian sinh ràng buộc yếu hơn nhật ký công bố.

- **Nhiễm dữ liệu**: biến thể đổi tên chỉ đổi định danh và xóa chú thích, cấu trúc mã giữ nguyên, nên không loại trừ hoàn toàn việc mô hình đã thấy các khuôn mẫu.

Chương 8. Kết luận và hướng phát triển

8.1 KẾT LUẬN

Tiểu luận đặt câu hỏi "grounded hay guessed" cho SemLoc [1], phương pháp định vị lỗi ngữ nghĩa neo lập luận của LLM vào ràng buộc thực thi được. Bằng cách tính lại từ nhật ký và chạy lại từ mã nguồn của tác giả, kiểm chứng lại bộ dữ liệu bằng thực thi, lặp lại thực nghiệm ba lần với kiểm định thống kê và đo đóng góp của từng tín hiệu, nhóm rút ra ba kết luận.

Thứ nhất, kết quả công bố tính lại được từ nhật ký, nhưng dòng đầu tiên của SemLoc — theo một lựa chọn thiết kế không có trong bài báo — luôn là dòng LLM tự đoán: Acc@1 của SemLoc (52,3%) là của dòng đoán, còn riêng phổ vi phạm ràng buộc chỉ đạt 43,2%. Bằng chứng thực thi có tác dụng thật, nhưng ở top-3.

Thứ hai, với các lỗi được kiểm thử thất bại chạy tới, đưa thẳng bằng chứng thực thi cho LLM — một prompt có kiểm thử thất bại và độ phủ từng dòng — chính xác hơn quy trình nhiều bước ở Acc@3, rẻ hơn khoảng năm lần và không thay đổi đáng kể khi đổi tên định danh; cách này không tìm được lỗi mà không kiểm thử nào chạy tới. Các kết luận này đến từ một mô hình chính, một bộ dữ liệu có nhãn đã sửa và 31 lỗi thật của một dự án.

Thứ ba, đánh giá định vị lỗi bằng LLM nhạy với cách chấm và với bộ dữ liệu: cách phá hòa, cách tính mẫu số và các dòng lỗi chuẩn sai đều làm thay đổi con số đáng kể. Phân tích thất bại cho thấy thêm rằng độ chính xác cấp dòng phụ thuộc vào việc dòng lỗi được gán ở dòng điều kiện hay ở câu lệnh mà điều kiện bảo vệ. Vì vậy, nhóm đề xuất mọi đánh giá như vậy áp dụng bảng kiểm tra bảy mục: phá hòa bằng hạng trung bình; mẫu số cố định; dòng lỗi chuẩn được kiểm chứng bằng thực thi; loại bỏ từng thành phần, kể cả thành phần không được mô tả; lặp lại và báo khoảng tin cậy ghép cặp; báo cáo chi phí; và so với một prompt duy nhất có cùng bằng chứng.

8.2 HƯỚNG PHÁT TRIỂN

Về nghiên cứu, bốn hướng mở ra từ kết quả: (i) bộ phán định dòng lỗi cho phép nhiều vị trí và chống các bản sửa "bù trừ" (làm kiểm thử đạt mà không sửa đúng chỗ lỗi), kèm phân xử thủ công các ca tranh chấp; (ii) một thí nghiệm loại bỏ tách riêng độ phủ khỏi kết quả kiểm thử trong prompt; (iii) đánh giá trên lỗi thật của nhiều dự án hơn và ở định vị cấp hàm trước khi xuống cấp dòng; (iv) một chuẩn đánh giá chung cho định vị lỗi bằng LLM, trong đó "một prompt có bằng chứng thực thi" là mốc so sánh bắt buộc.

Phụ lục A. Hướng dẫn tái hiện và bảng bổ sung

A.1 BỐN MỨC TÁI HIỆN

A.1.1 Từ dữ liệu đến bảng

Toàn bộ mã nguồn, dữ liệu dẫn xuất và hướng dẫn nằm tại `https://github.com/doanan293/grounded-or-guessed` (tệp `REPRODUCE.md`). Bốn mức, từ rẻ đến đắt:

1. **Mức 0** — sinh lại bảng, hình và macro số liệu từ các tệp CSV đã có: `python scripts/make_tables.py` và `python scripts/make_numbers.py`.
2. **Mức 1** — sinh lại mọi tệp CSV từ gói kết quả các lượt chạy: giải nén gói rồi chạy `scripts/regenerate_all.sh`; trong Docker, các tệp CSV và bảng sinh ra trùng từng byte với bản đã công bố.
3. **Mức 2** — chạy lại chính SemLoc và các phương pháp đối chứng, phát lại mọi câu trả lời LLM từ bộ đệm, không gọi mạng (`FLX_LLM_OFFLINE=1`).
4. **Mức 3** — chạy mới qua một điểm cuối LLM tương thích OpenAI, với mã lượt chạy mới.

A.1.2 Môi trường

Ảnh Docker (`docker build -t grounded-or-guessed .`) chứa Python 3.12 và toàn bộ phụ thuộc, đặt chế độ không mạng cho LLM; lệnh mặc định chạy bộ kiểm thử đơn vị. Gói kết quả các lượt chạy (đầu ra SemLoc, nhật ký và bộ đệm lượt gọi LLM, bảng băm kiểm tra) được tạo bằng `scripts/export_artifact.py`; công cụ từ chối đóng gói nếu phát hiện khóa truy cập hay địa chỉ thư điện tử.

A.2 BẢNG BỔ SUNG

A.2.1 Phân rã trên nhật ký công bố

Bảng A.1: Phân rã Shapley trên nhật ký công bố của tác giả (dòng lỗi chuẩn gốc)

Cấu hình	Tập con	Chỉ số	SemLoc	Điểm thưởng dòng đoán	Phổ ngữ nghĩa
SemLoc, Claude Sonnet 4,6, $T{=}0,8$	tất cả	Acc@1	42,8	27,8 [22,5; 33,5]	15,0 [12,1; 18,0]
SemLoc, Claude Sonnet 4,6, $T{=}0,8$	tất cả	Acc@3	65,6	26,4 [21,9; 31,4]	39,2 [34,2; 44,1]
SemLoc, Claude Sonnet 4,6, $T{=}0,8$	thực thi được	Acc@1	57,8	37,6 [31,0; 44,3]	20,3 [16,3; 24,3]
SemLoc, Claude Sonnet 4,6, $T{=}0,8$	thực thi được	Acc@3	77,3	35,7 [30,0; 41,6]	41,6 [36,3; 47,2]
SemLoc, Gemini 2,5 Pro, $T{=}0,8$	tất cả	Acc@1	39,6	26,0 [21,1; 31,3]	13,6 [10,8; 16,6]
SemLoc, Gemini 2,5 Pro, $T{=}0,8$	tất cả	Acc@3	64,4	24,2 [20,0; 28,9]	40,2 [34,7; 45,4]
SemLoc, Gemini 2,5 Pro, $T{=}0,8$	thực thi được	Acc@1	53,5	35,1 [28,6; 42,2]	18,4 [14,7; 22,1]
SemLoc, Gemini 2,5 Pro, $T{=}0,8$	thực thi được	Acc@3	73,0	32,7 [27,3; 38,5]	40,3 [34,8; 45,6]

A.2.2 Prompt của phương pháp đối chứng LLM + độ phủ

Mỗi dòng mã được chú thích `[f=<số kiểm thử thất bại chạy qua>/p=<số kiểm thử thành công chạy qua>]`; thông điệp hệ thống yêu cầu trả lời đúng định dạng JSON:

```
You are a fault-localization assistant. Reply with JSON only:
{"lines": [<line numbers, most suspicious first, at most 5>]}

The program below has exactly one faulty line. <n_fail> tests fail and
   <n_pass> pass.
Each line is annotated with [f=<number of failing tests that execute it
   >/p=<number of passing tests>].
<numbered source with annotations>
Failing tests:
- <test name>: <first error line>
Return the five most suspicious line numbers, most suspicious first.
```

Tài liệu tham khảo

[1] Z. Yang, H. Zhu, Q. Zhang, R. Gupta, and A. Kundu, “Semloc: Structured grounding of free-form llm reasoning for fault localization,” 2026. [Online]. Available: https://arxiv.org/abs/2603.29109

[2] H. Yang, Y. Peng, and Z. Ding, “Effiholmes: Differential profiling-guided repository level time inefficiency fix localization,” 2026. [Online]. Available: https://arxiv.org/abs/2608.03558

[3] M. Medeiros, U. Kulesza, C. Treude, D. Lucena, R. Gomes, R. Coelho, E. Adachi, and R. Bonifacio, “Integrating crash report mining and llms for bug localization and repair: An industrial report,” 2026. [Online]. Available: https://arxiv.org/abs/2609.04483

[4] J. A. Jones, M. J. Harrold, and J. Stasko, “Visualization of test information to assist fault localization,” in *Proceedings of the 24th international conference on Software engineering - ICSE ’02*, ser. ICSE ’02. ACM Press, 2002, p. 467. [Online]. Available: http://dx.doi.org/10.1145/581396.581397

[5] R. Abreu, P. Zoeteweij, R. Golsteijn, and A. J. van Gemund, “A practical evaluation of spectrum-based fault localization,” *Journal of Systems and Software*, vol. 82, no. 11, p. 1780–1792, Nov. 2009. [Online]. Available: http://dx.doi.org/10.1016/j.jss.2009.06.035

[6] W. E. Wong, R. Gao, Y. Li, R. Abreu, and F. Wotawa, “A survey on software fault localization,” *IEEE Transactions on Software Engineering*, vol. 42, no. 8, p. 707–740, Aug. 2016. [Online]. Available: http://dx.doi.org/10.1109/TSE.2016.2521368

[7] S. Kang, G. An, and S. Yoo, “A quantitative and qualitative evaluation of llm-based explainable fault localization,” 2023. [Online]. Available: https://arxiv.org/abs/2308.05487

[8] Y. Qin, S. Wang, Y. Lou, J. Dong, K. Wang, X. Li, and X. Mao, “Agentfl: Scaling llm-based fault localization to project-level context,” 2024. [Online]. Available: https://arxiv.org/abs/2403.16362

[9] C. Xu, Z. Liu, X. Ren, G. Zhang, M. Liang, and D. Lo, “Flexfl: Flexible and effective fault localization with open-source large language models,” 2024. [Online]. Available: https://arxiv.org/abs/2411.10714

[10] J. Sallou, T. Durieux, and A. Panichella, “Breaking the silence: the threats of using llms in software engineering,” 2023. [Online]. Available: https://arxiv.org/abs/2312.08055

[11] D. Ramos, C. Mamede, K. Jain, P. Canelas, C. Gamboa, and C. L. Goues, “Are large language models memorizing bug benchmarks?” 2024. [Online]. Available: https://arxiv.org/abs/2411.13323

[12] R. Widyasari, S. Q. Sim, C. Lok, H. Qi, J. Phan, Q. Tay, C. Tan, F. Wee, J. E. Tan, Y. Yieh, B. Goh, F. Thung, H. J. Kang, T. Hoang, D. Lo, and E. L. Ouh, “Bugsinpy: a database of existing bugs in python programs to enable controlled testing and debugging studies,” in *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE ’20. ACM, Nov. 2020, p. 1556–1560. [Online]. Available: http://dx.doi.org/10.1145/3368089.3417943

[13] W. E. Wong, V. Debroy, R. Gao, and Y. Li, “The dstar method for effective software fault localization,” *IEEE Transactions on Reliability*, vol. 63, no. 1, p. 290–308, Mar. 2014. [Online]. Available: http://dx.doi.org/10.1109/TR.2013.2285319

[14] H. A. de Souza, M. L. Chaim, and F. Kon, “Spectrum-based software fault localization: A survey of techniques, advances, and challenges,” 2016. [Online]. Available: https://arxiv.org/abs/1607.04347

[15] M. Papadakis and Y. Le Traon, “Metallaxis-fl: mutation-based fault localization,”

Software Testing, Verification and Reliability, vol. 25, no. 5-7, p. 605–628, Sep. 2013. [Online]. Available: http://dx.doi.org/10.1002/stvr.1509

[16] S. Moon, Y. Kim, M. Kim, and S. Yoo, "Ask the mutants: Mutating faulty programs for fault localization," in *2014 IEEE Seventh International Conference on Software Testing, Verification and Validation*. IEEE, Mar. 2014, p. 153–162. [Online]. Available: http://dx.doi.org/10.1109/ICST.2014.28

[17] A. Zeller and R. Hildebrandt, "Simplifying and isolating failure-inducing input," *IEEE Transactions on Software Engineering*, vol. 28, no. 2, p. 183–200, Feb. 2002. [Online]. Available: http://dx.doi.org/10.1109/32.988498

[18] A. Zeller, "Isolating cause-effect chains from computer programs," *ACM SIGSOFT Software Engineering Notes*, vol. 27, no. 6, p. 1–10, Nov. 2002. [Online]. Available: http://dx.doi.org/10.1145/605466.605468

[19] M. D. Ernst, J. H. Perkins, P. J. Guo, S. McCamant, C. Pacheco, M. S. Tschantz, and C. Xiao, "The daikon system for dynamic detection of likely invariants," *Science of Computer Programming*, vol. 69, no. 1-3, p. 35–45, Dec. 2007. [Online]. Available: http://dx.doi.org/10.1016/j.scico.2007.01.015

[20] R. Just, D. Jalali, and M. D. Ernst, "Defects4j: a database of existing faults to enable controlled testing studies for java programs," in *Proceedings of the 2014 International Symposium on Software Testing and Analysis*, ser. ISSTA '14. ACM, 2014, p. 437–440. [Online]. Available: http://dx.doi.org/10.1145/2610384.2628055

[21] D. Lin, J. Koppel, A. Chen, and A. Solar-Lezama, "Quixbugs: a multi-lingual program repair benchmark set based on the quixey challenge," in *Proceedings Companion of the 2017 ACM SIGPLAN International Conference on Systems, Programming, Languages, and Applications: Software for Humanity*, ser. SPLASH '17. ACM, Oct. 2017, p. 55–56. [Online]. Available: http://dx.doi.org/10.1145/3135932.3135941

[22] S. Pearson, J. Campos, R. Just, G. Fraser, R. Abreu, M. D. Ernst, D. Pang, and B. Keller, “Evaluating and improving fault localization,” in *2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE)*. IEEE, May 2017, p. 609–620. [Online]. Available: http://dx.doi.org/10.1109/ICSE.2017.62

[23] C. Xu, S. Guan, D. Greene, and M.-T. Kechadi, “Benchmark data contamination of large language models: A survey,” 2024. [Online]. Available: https://arxiv.org/abs/2406.04244